

Orden y estado global

Sistemas Distribuidos



UNIVERSIDAD AUTÓNOMA METROPOLITANA
ELIZABETH PÉREZ CORTÉS - UNIDAD IZTAPALAPA

Contenido

- 4.1 Relojes lógicos escalares**
- 4.2 Relojes lógicos vectoriales**
- 4.3 Estado global**

Contenido

- 
- 4.1 Relojes lógicos escalares**
 - 4.2 Relojes lógicos vectoriales**
 - 4.3 Estado global**

Problema

- En los sistemas asíncronos no hay reloj común
- Dado un conjunto de peticiones de acceso a una sección crítica
 - ¿En qué orden se ejecutaron las peticiones?
 - ¿A quién se le atiende primero?
- Necesitamos medios para **ordenar** las peticiones
- Para poder decir que un evento ocurrió antes que otro



Modelo

- Conjunto de procesadores
- Sin memoria común
- Sin reloj común
- Comunicación a través de mensajes finitos enviados por canales sin pérdida y con retardo finito pero impredecible

Modelo

Sobre esta infraestructura se tiene:

- Una colección de procesos
- Cada proceso P_i es una secuencia de eventos
- Los eventos pueden ser:
 - internos,
 - envío o
 - recepción.
- En un proceso se dice que un evento A ocurre antes que B si en la secuencia que define el proceso, A aparece antes que B
- Cada proceso se puede ver como un conjunto de eventos con un orden total definido sobre ellos.

Relación “sucedió antes”

- **Definición:** La relación “ $\rightarrow$ ” sobre el conjunto de eventos de un sistema es la que satisface las siguientes condiciones:
 1. Si a y b son eventos en el mismo proceso, y a viene antes que b , entonces $a \rightarrow b$
 2. Si a es el envío de un mensaje y b es la recepción de ese mismo mensaje por otro proceso, entonces $a \rightarrow b$
 3. Si $a \rightarrow b$ y $b \rightarrow c$ entonces $a \rightarrow c$

Concurrencia

- **Definición:** Dos eventos distintos a y b , se dice que son **concurrentes** si a no sucedió antes que b “ $\sim(a \rightarrow b)$ ” y b no sucedió antes que a “ $\sim(b \rightarrow a)$ ”.
- Se asume que a no sucedió antes que a “ $\sim(a \rightarrow a)$ ”.
- La relación sucedió antes “ $\rightarrow$ ” es un orden parcial sobre los eventos en el sistema

$a \rightarrow b$ Es equivalente a poder viajar entre a y b en la representación gráfica

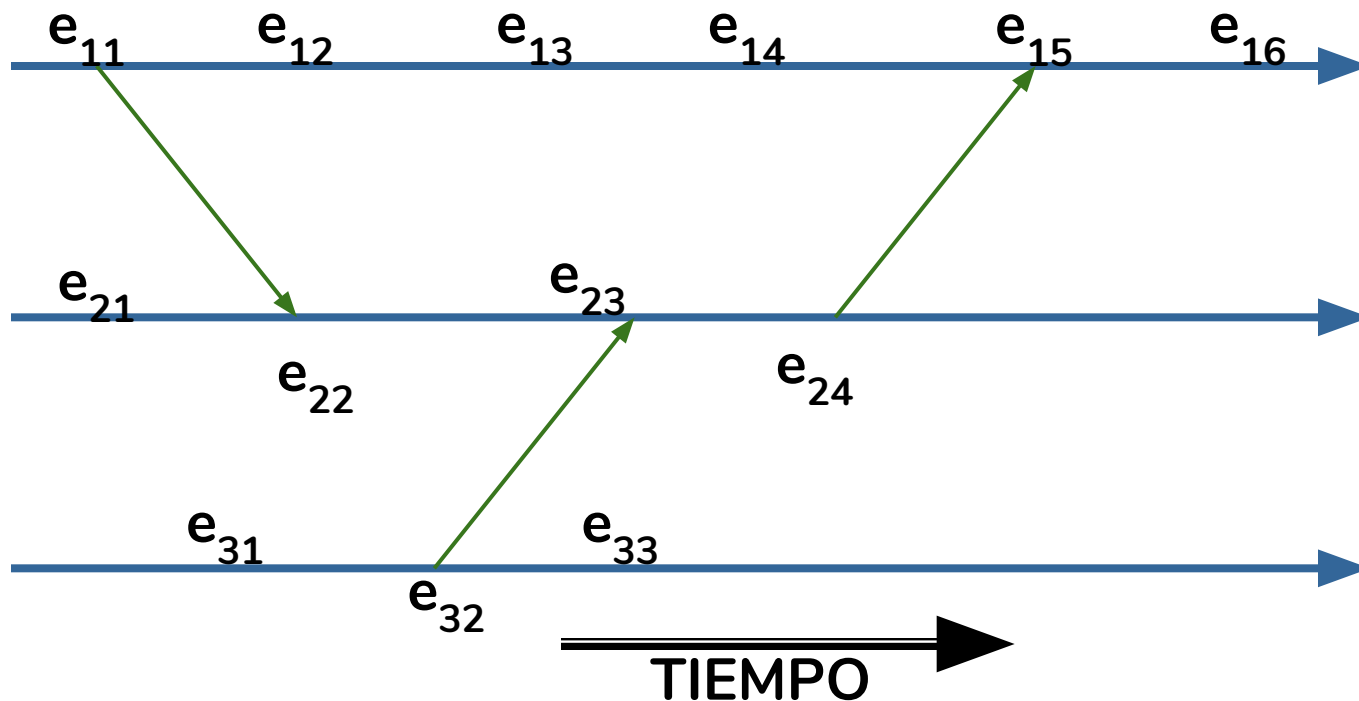
PROCESOS

P1

Ejemplo: $e_{13} \rightarrow e_{14}$, $e_{11} \rightarrow e_{22}$ y $e_{31} \rightarrow e_{16}$

P2

P3

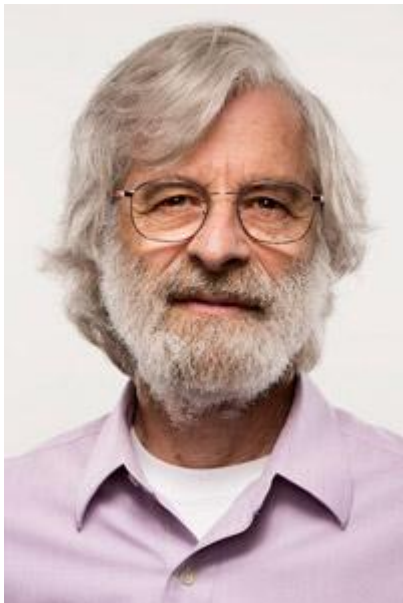


Causalidad

- Otra manera de leer la relación $\rightarrow$ es decir que $a \rightarrow b$ significa que es posible para el evento a afectar causalmente el evento b .
- Dos eventos son concurrentes si ninguno de los dos pudo afectar al otro.
- Ejemplo: los eventos e_{13} y e_{32}

Relojes lógicos - Leslie Lamport

- **Premio Turing 2013:** Coherencia en sistemas distribuidos.



"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable"

Why Don't Computer Scientists Learn Math?

$\{f \in [1..N \rightarrow 1..N] : \forall y \in 1..N : \exists x \in 1..N : f[x]=y\}$

Relojes lógicos

(Lamport CACM 7,78)

- a) Para cada proceso P_i hay un reloj R_i
 - b) R_i es una función que le asigna un número $R_i(a)$ a todo evento en el proceso P_i
 - c) El sistema de relojes global se puede ver como R tal que $R(a) = R_i(a)$ si a es un evento del proceso P_i
- ¿Cuándo es correcto este sistema?



Condición de reloj

- Condición de reloj: Para cualquier par de eventos a, b :

Si $a \rightarrow b$ entonces $R(a) < R(b)$

- Esta condición se satisface si se satisfacen las dos condiciones siguientes:

1) Si a y b son eventos en el proceso P_i y a sucede antes que b , entonces $R_i(a) < R_i(b)$

Es decir, el orden dado por el reloj es compatible con el orden local

Condición de reloj

Y

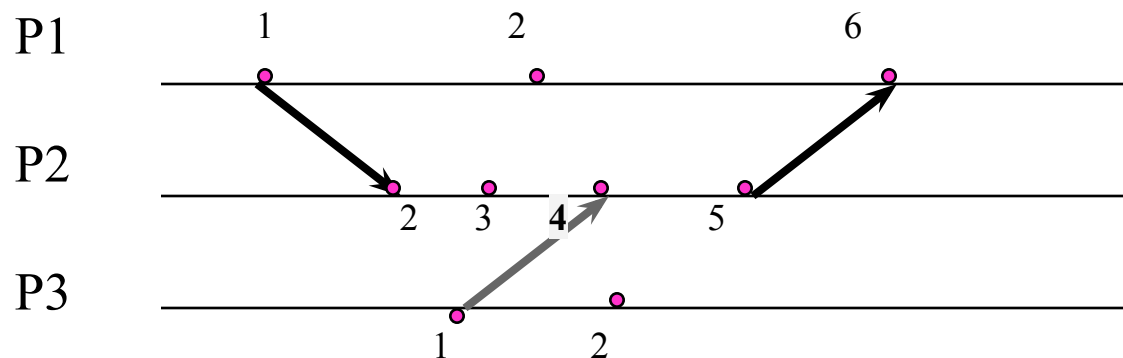
2) Si a es el envío de un mensaje de P_i y b es la recepción del mismo mensaje en P_j entonces $R_i(a) < R_j(b)$

Es decir, el orden es compatible con la causalidad de la comunicación

Un orden que satisface estas dos condiciones es un orden CAUSAL

Relojes lógicos escalares

- Ejemplo:



Los eventos con la misma fecha son causalmente dependientes

Implementación: Cumplimiento de condición 1

La condición 1 se satisface con la siguiente regla de implementación:

Regla implementación 1: Cada proceso P_i incrementa R_i entre dos eventos sucesivos

Implementación: Cumplimiento de condición 2

Para que la condición 2 se cumpla es necesario que:

- Al enviarse, el mensaje es marcado con una estampilla de tiempo T_m igual al tiempo en el que el mensaje fue enviado
- A la recepción de un mensaje marcado con T_m , el receptor avance su reloj para que sea más grande que T_m .

Implementación: Cumplimiento de condición 2

Más precisamente, se tiene la siguiente regla de implementación:

Regla de implementación 2:

- a) Si un evento a es el envío de un mensaje m por el proceso P_i , entonces el mensaje contiene una estampilla de tiempo $T_m = R_i(a)$
- b) A la recepción de un mensaje m , el proceso P_j actualiza su reloj $R_j = \text{Max}(T_m, R_j)$

Relojes lógicos escalares

(Lamport CACM 7,78)

- a) Para cada proceso P_i hay un reloj R_i
- b) La fecha de un evento es la hora de su reloj
- c) Al inicio todos los relojes tienen una fecha inicial $R_i = 0$
- d) Si se produce un evento local e en el sitio i

$$R_i = R_i + 1$$

$$\text{fecha}(e) = R_i$$

- e) Todo mensaje tiene la forma (m, H) donde m es el contenido y H la hora de su emisión
- f) Cuando i recibe un mensaje: Si $R_i < H$ entonces $R_i = H$

$$R_i = R_i + 1$$

Y esa es la hora que se le asigna al evento de recepción

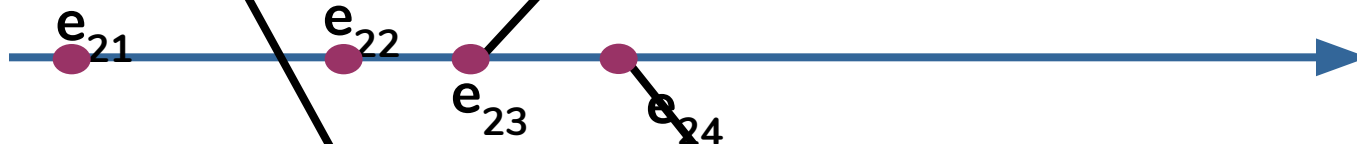


Ejemplo: Ponga fecha a los eventos

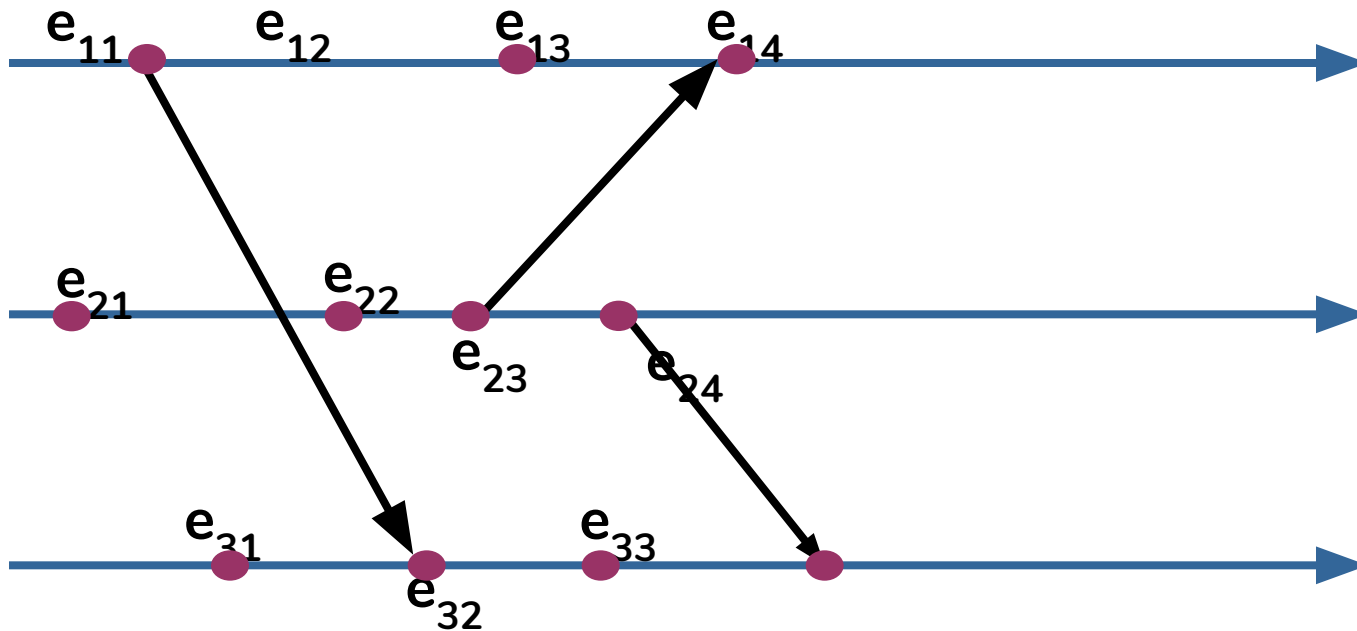
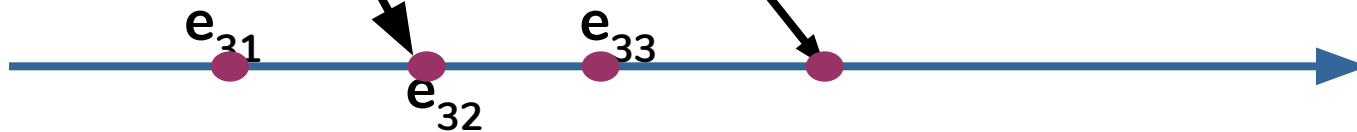
P1



P2



P3

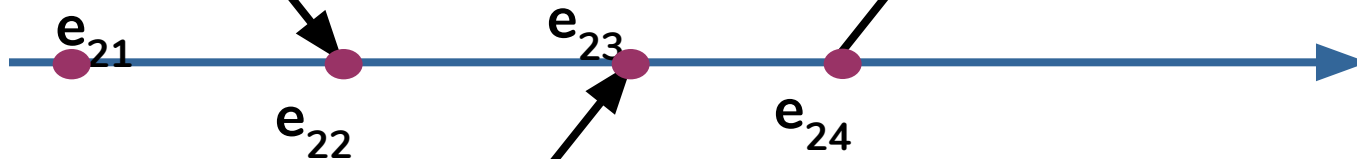


Ejercicio: Ponga fecha a los eventos

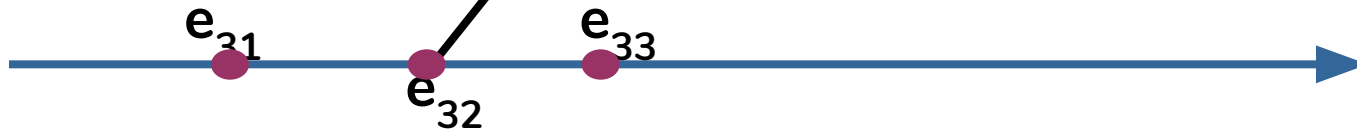
P1



P2



P3

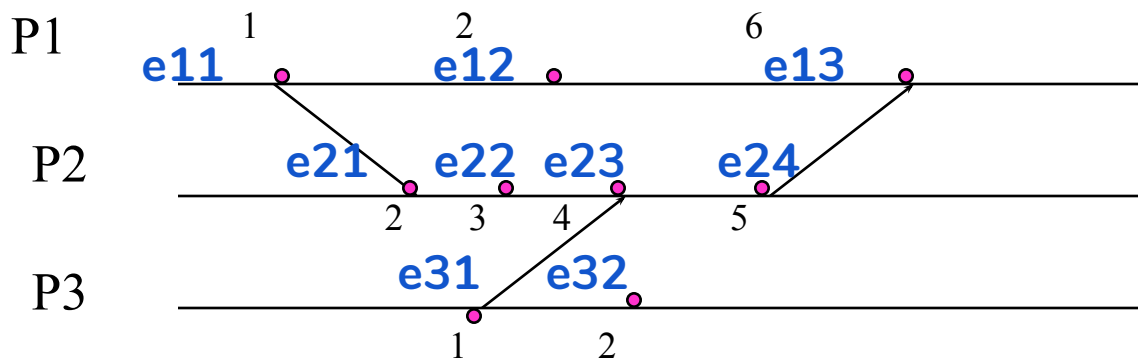


Orden total usando los relojes lógicos escalares

- Se ordenan los eventos de acuerdo a sus estampa de tiempo y para romper los empates se usa un orden entre los sitios. En otros términos
- Si a es un evento en el proceso P_i y b es un evento en el proceso P_j , entonces $a \Rightarrow b$ (a precede a b) si:
 - i) $R_i(a) < R_j(b)$ ó
 - ii) $R_i(a) = R_j(b)$ y $P_i < P_j$

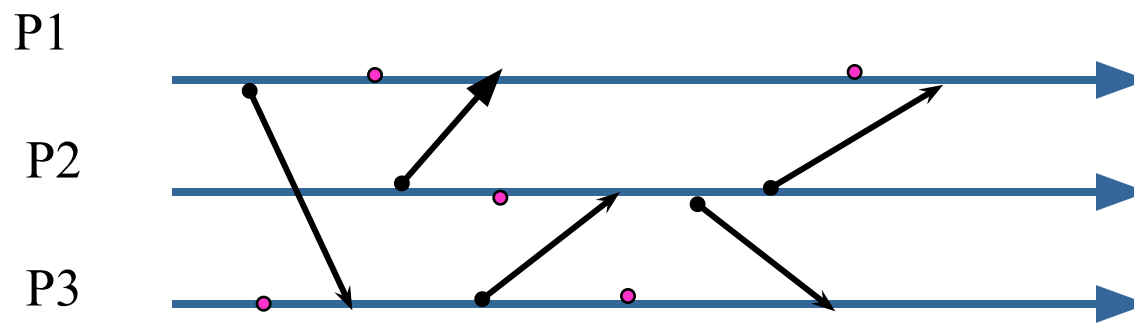
Relojes lógicos: Orden total

- Ejemplo: Nombrando e_{ij} al evento j del sitio i



Orden total: e_{11} e_{31} e_{12} e_{21} e_{32} e_{22} e_{23} e_{24} e_{13}

Ejercicio relojes lógicos



Nombre los eventos, asigne fechas lógicas escalares y ordénelos totalmente.

Ejemplo de uso Exclusión mutua



Ejemplo de uso: Exclusión mutua

- Considere un sistema de n procesos
 - $P_1, P_2, \dots, P_n$
- Los segmentos de código en los cuales un proceso manipula un recurso compartido de uso exclusivo se denominan **secciones críticas**.
- Para no tener resultados erróneos, la sección crítica se debe ejecutar en *exclusión mutua*: es decir, sólo un proceso a la vez está ejecutando su sección crítica.

Conceptos básicos: Exclusión mutua

- Una solución al problema de la sección crítica debe cumplir los siguientes requisitos
 - Exclusión mutua: Si P_i está ejecutando su sección crítica, ningún otro proceso está ejecutando su sección crítica
 - Progreso: Si nadie está ejecutando su sección crítica se debe elegir en un tiempo limitado quien entra a hacerlo
 - Espera limitada: Nadie debe esperar indefinidamente por entrar en la sección crítica. Es decir se debe evitar la inanición (starvation)

Conceptos básicos: Exclusión mutua

- En toda solución hay una fase de entrada en donde se solicita el permiso de ejecución y una fase de salida en donde se notifica al sistema la finalización de la sección crítica.

ADQUIERE DERECHO DE ENTRADA

EJECUTA SECCIÓN CRÍTICA

NOTIFICA TERMINACIÓN

Interbloqueos

Un interbloqueo (*deadlock*) se produce cuando en un sistema se cumplen las siguientes condiciones simultáneamente:

1. **Exclusión mutua:** los recursos deben ser usados en exclusión mutua, es decir, o están asignados a un proceso o están libres.
2. **Retención y espera:** un proceso que tiene recursos asignados puede solicitar otro recurso y, sin liberar los que ya tiene, esperar a que le sea asignado.

Interbloqueos

3. No expropiación: el recurso se libera hasta que el proceso al que se le asignó lo termine de utilizar, es decir, no es posible quitárselo para asignarlo a otro proceso.
4. Espera circular: existe un conjunto de procesos $\{p_1, p_2, \dots, p_n\}$ con $n \geq 2$ tal que p_i espera un recurso que tiene asignado p_{i+1} para i en $[1, n-1]$ y p_n espera por un recurso que tiene asignado p_1 .

Modelo de sistema

- Asíncrono
- N nodos que operan correctamente
- Comunicación por mensajes (sin memoria compartida)
- Canales sin pérdida, sin errores, sin garantía de orden y con tiempo de entrega variable.

Algoritmo

- Cada nodo n_i tiene 3 procesos para atender la exclusión mutua. Uno para cada uno de los siguientes eventos:
 1. La solicitud local de entrada a la sección crítica
 2. La solicitud de entrada a la sección crítica de otro p_j
 3. La respuesta de otro p_j a la solicitud del nodo p_i .

Algoritmo

- Esos procesos tienen dos constantes
 me : el identificador del nodo
 N : el número de nodos en el sistema
- Y usan las siguientes variables compartidas:
 $T_{petición} = 0$;
 $Valor_mas_alto_recibido = 0$;
 $Respuestas_pendientes = N - 1$;
 $Solicita_EM = Falso$ /*Indicador de solicitud de entrada a la sección crítica pendiente */
 $SolicitudPendiente[N] = \{Falso, ..., Falso\}$

Algoritmo

- Y un semáforo para protegerlas

Mutex = 0 /* el derecho de acceso está libre */
/* Se omite el uso de este semáforo
para clarificar el algoritmo */

- Los nodos intercambian dos tipos de mensajes

1. **REQUEST** : para notificar a los nodos su requerimiento de entrar a la sección crítica
2. **REPLY**: para notificar a un nodo solicitante que puede entrar a la sección crítica

Ejecución de la sección crítica en P_i

```
...  
DLock()  
    SECCIÓN CRÍTICA  
DUnlock()  
...
```

DLOCK()

PROCEDIMIENTO DLock()

COMIENZA

Solicita_EM = Verdad;

Tpetición = Valor_mas_alto_recibido + 1;

Respuestas_pendientes = N-1;

/ Se notifica a los demás */*

Para k = 1 hasta N haz

 Si k ≠ i ENTONCES

 envía(REQUEST(Tpetición, i), k)

/ Esperamos el acuerdo de todos */*

Espera hasta que Respuestas_pendientes == 0

TERMINA

UNLOCK()

PROCEDIMIENTO DUNLOCK()

COMIENZA

 Solicita_EM = FALSO;

 Para k = 1 HASTA N Haz

 Si (SolicitudPendiente[k]) ENTONCES
 envía(REPLY, k);
 SolicitudPendiente[k] = Falso;

 FINSI

FINPARA

TERMINA

REQUEST

Al recibir REQUEST(T_j, j) en P_i

```

Valor_mas_alto_recibido = MAX(Valor_mas_alto_recibido,  $T_j$ )
Si (Solicita_EM = FALSO) Entonces
    ENVIA(REPLY,  $j$ );
Otro
    Si ( $T_{petición,i}$  > ( $T_j, j$ ) Entonces /* $P_j$  llegó antes*/
        ENVIA(REPLY,  $j$ ),
    Otro SolicitudPendiente[ $j$ ] = VERDAD;
  
```

REPLY

Al recibir REPLY de P_j a la solicitud de p_i
RespuestasPendientes -= 1

Suposición

- Los accesos a las variables compartidas estarán debidamente protegidos
- Sólo se hace una petición por sitio en un instante dado y hasta que esa termina se hace otra

Ejemplo de ejecución: 3 solicitudes

<T=1, P1>

Los relojes lógicos cuentan solicitudes de entrada

<T=2, P3>

<T=1, P4>

➤ REQUEST
➤ REPLY

P2 contesta a todos pues él no quiere entrar

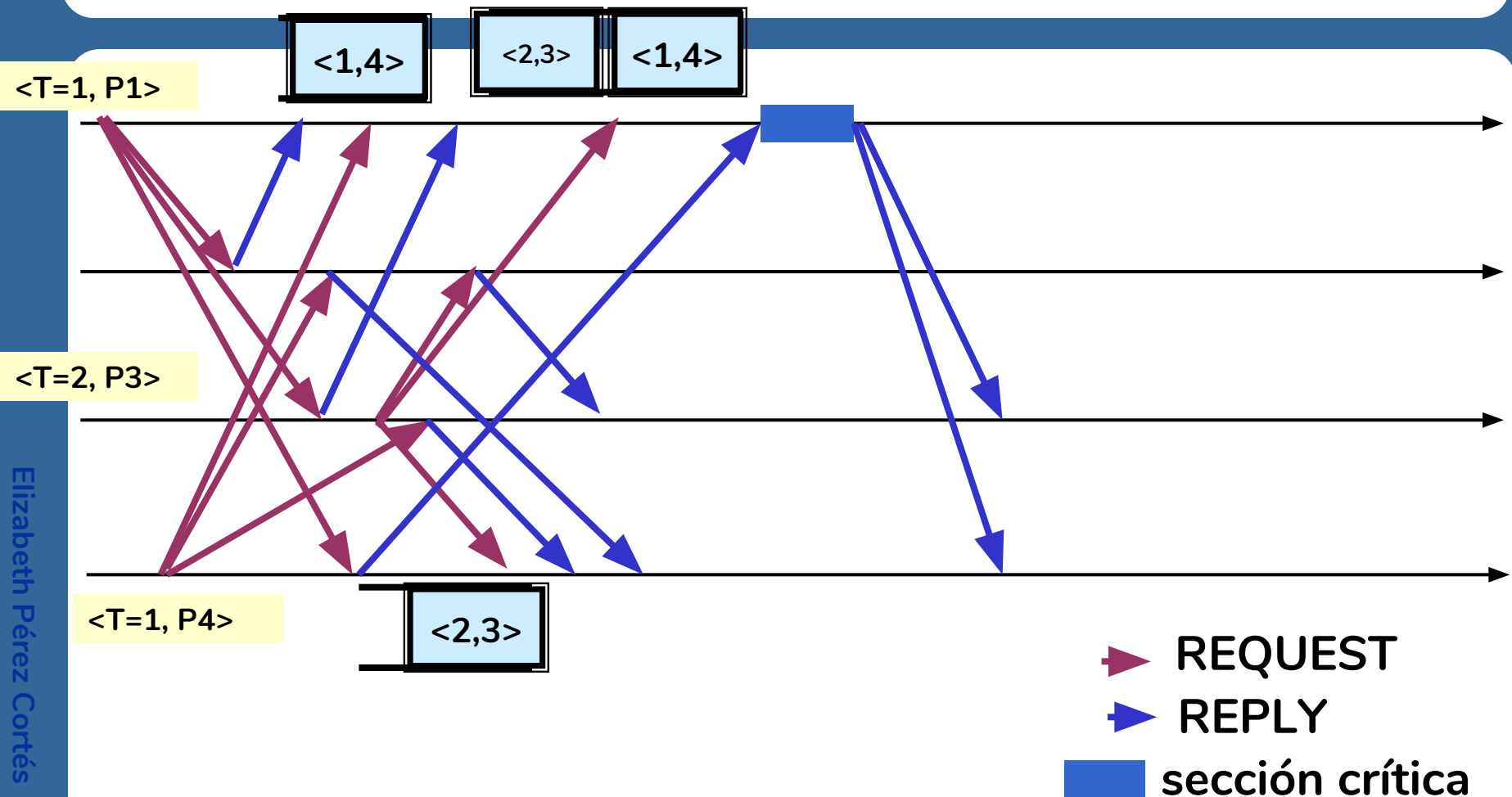
<T=1, P1>

<T=2, P3>

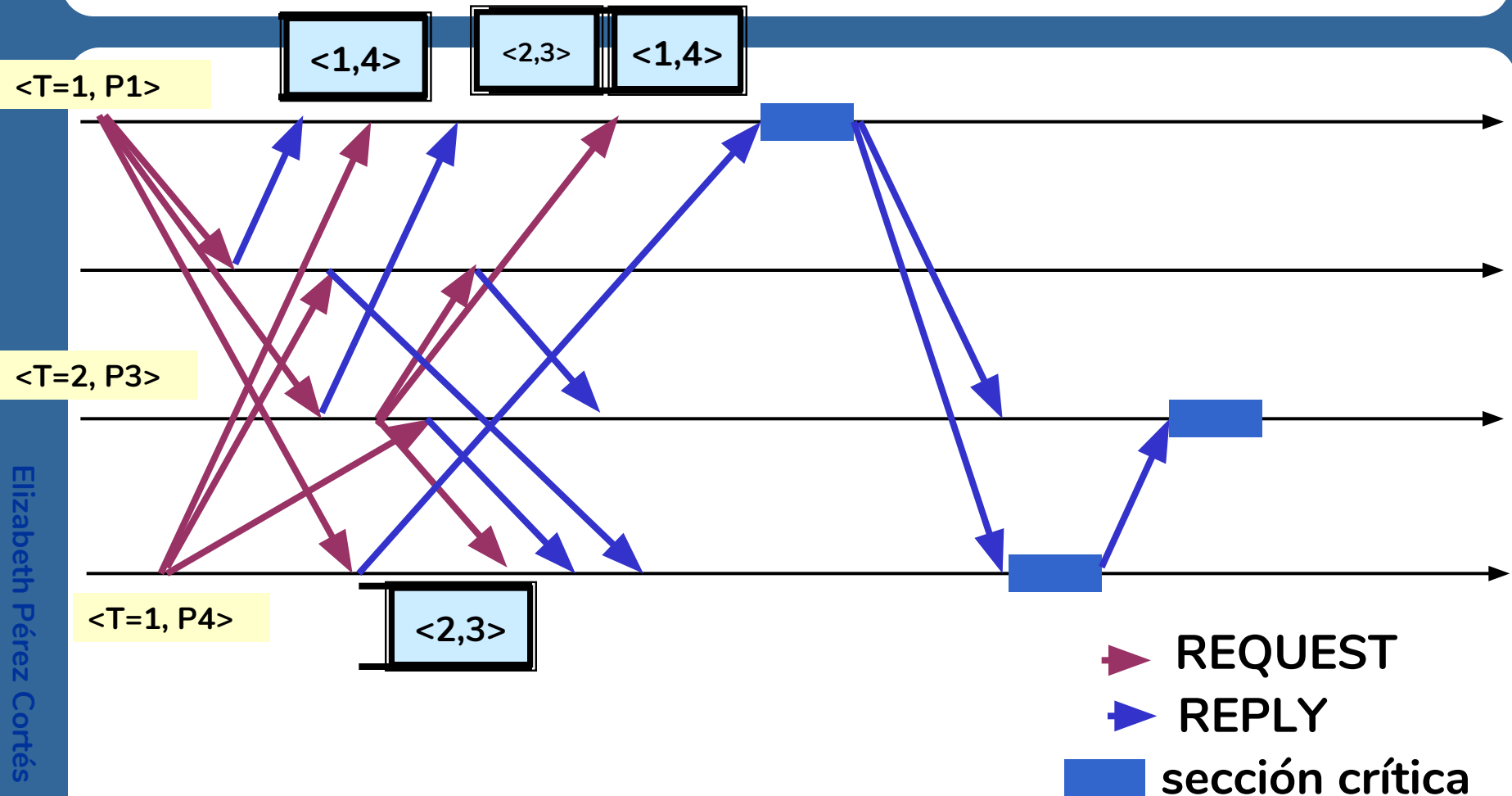
<T=1, P4>

➤ REQUEST
➤ REPLY
■ sección crítica

P1 ya tiene todas las respuestas, ejecuta su SC y al salir le contesta a P4 y a P3



P4 ejecuta su SC y al salir le contesta a P3 que finalmente ejecuta su SC



Complejidad del algoritmo

- Memoria: lineal $O(n)$
- Mensajes: Por cada solicitud de acceso a la Sección crítica se necesitan:
 - N-1 mensajes REQUEST para solicitar
 - N-1 mensajes REPLY para poder entrar
 - En total: $2(N-1)$ mensajes en cualquier caso.

El algoritmo es correcto

- **Asegura exclusión mutua:**
 - Examine el caso en el que esto sería posible, asuma que dos procesos A y B se dan mutuamente REPLY y esto lleva a una contradicción
- **Asegura progreso:** No es posible el deadlock porque no es posible la espera circular pues el orden de las peticiones es total.
- **Asegura espera limitada:** la petición del nodo que espera será en algún momento la del número más pequeño porque cualquier nueva solicitud tendrá un número mayor que la de él.

Contenido

4.1 Relojes lógicos escalares

 4.2 Relojes lógicos vectoriales

4.3 Estado global

Motivación Relojes vectoriales

Con los relojes lógicos se le asocian fechas a los eventos y sabemos que si:

$e \rightarrow e'$ entonces $R(e) < R(e')$

$e' \rightarrow e$ entonces $R(e') < R(e)$

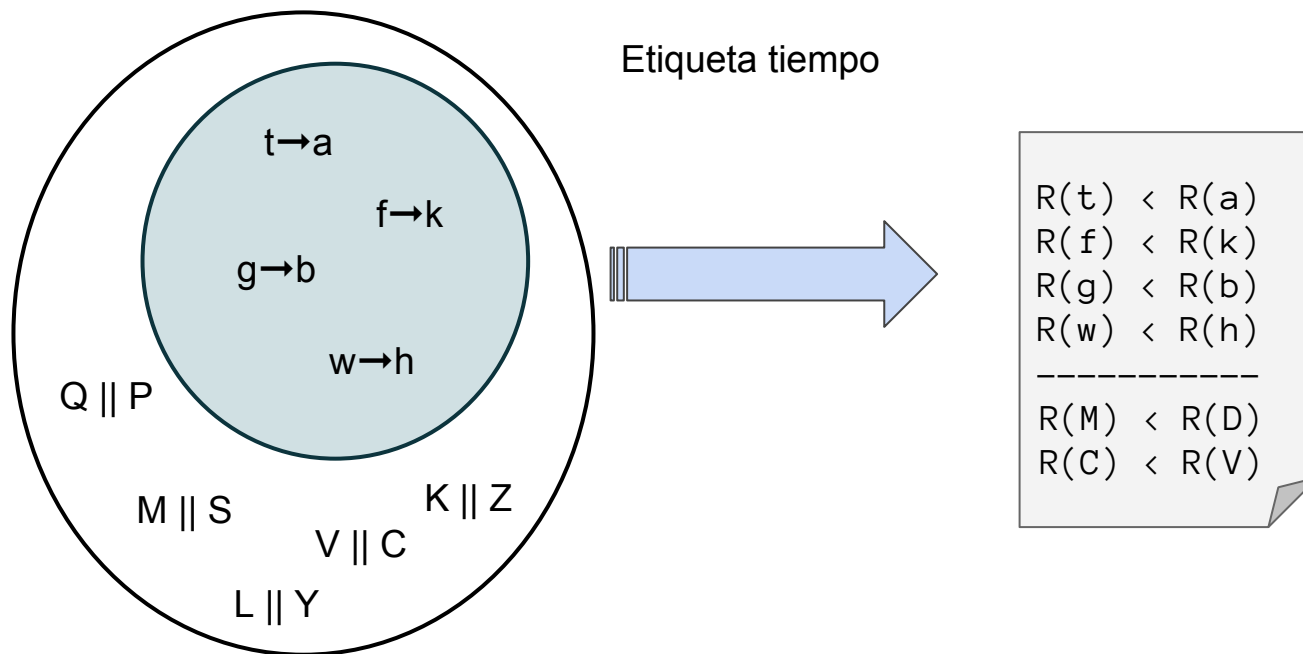
$e' \parallel e$ ¿¿¿???

Si un sistema de relojes R cumple:

- $e1 \rightarrow e2 \Rightarrow R(e1) < R(e2)$, se dice que exhiben consistencia débil
- $e1 \rightarrow e2 \Leftrightarrow R(e1) < R(e2)$, se dice que exhiben consistencia fuerte

Los relojes lógicos escalares sólo son débilmente consistentes.

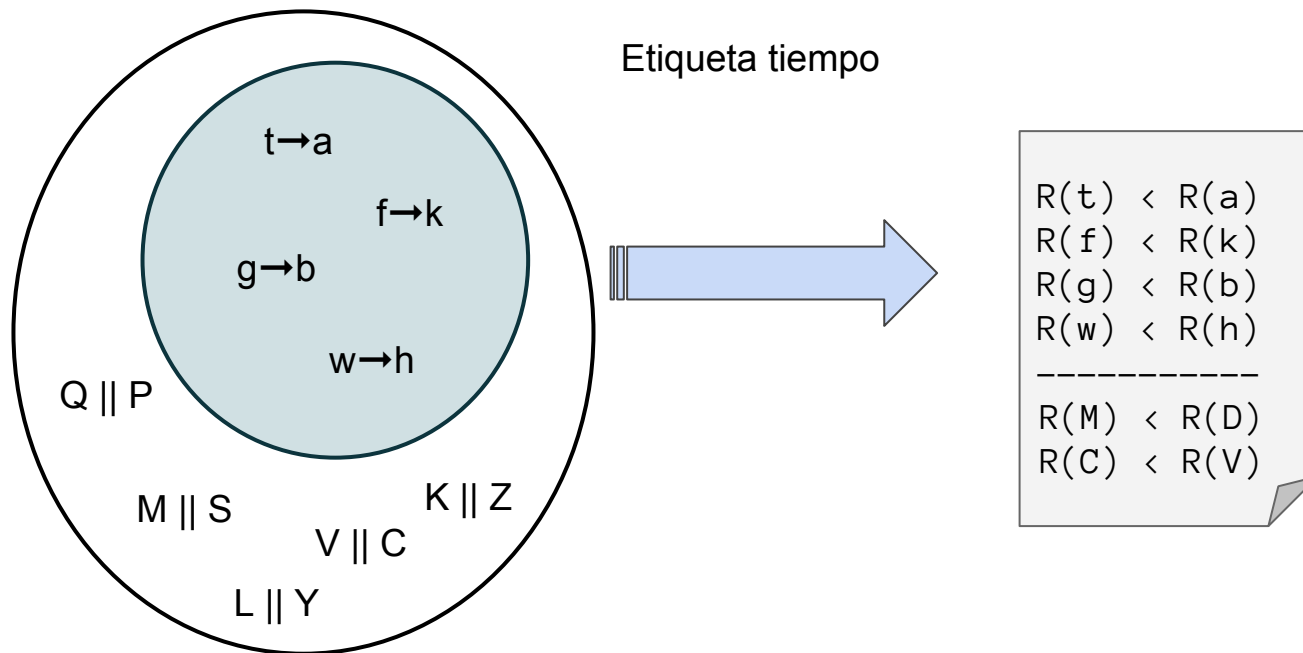
Consistencia débil VS fuerte



Consistencia débil: Si el evento a *sucedio antes que* b , es seguro que sus tiempos serán $R(a) < R(b)$.

Pero, es posible que para *eventos concurrentes* A y B , también pase que $R(A) < R(B)$.

Consistencia débil VS fuerte



Consistencia fuerte:

Si el evento *a* *sucedió antes que* *b*, seguro que $R(a) < R(b)$.



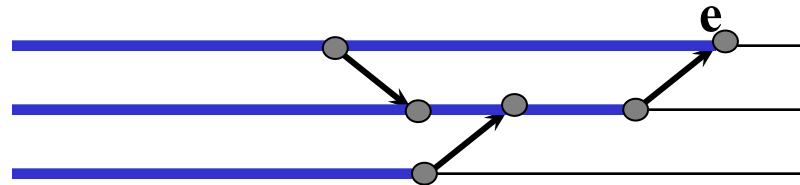
Además...

Si tenemos que $R(a) < R(b)$, es seguro que *a* *sucedió antes que* *b*.



Motivación Relojes vectoriales

El orden de los relojes lógicos escalares no conservan el pasado de un evento como se puede observar en la siguiente figura:



El reloj lógico escalar sólo me dice cuánto mide la cadena más larga de causalidad que precede al evento.

Relojes vectoriales

- La historia de un evento se define como el conjunto de eventos que lo preceden causalmente

$$h(e) = \{a \mid \text{tal que } a \rightarrow e\}$$

$$h_i(e) = \{a \mid \text{tal que } a \rightarrow e \text{ y } a \text{ es un evento de } P_i\}$$

$$\text{entonces } a \rightarrow e \quad \text{ssi} \quad h(a) \subset h(e)$$

- Con los relojes vectoriales cada vez que se envía un mensaje se envía su historia. La historia se puede representar enviando el número más grande de evento en cada P_i .

Relojes vectoriales

- Un reloj vectorial de un evento e que ocurre en P_i se define como:

$$V(e) = [v_1, v_2, v_3, \dots, v_n]$$

donde $v_j = |h_j(e)|$ para $i \neq j$ y $v_i = |h_i(e)| + 1$

- Cada nodo i conserva un reloj vectorial propio V_i
- Estado inicial:
 - $V_i[j] = 0$ para todo i, j

Relojes vectoriales

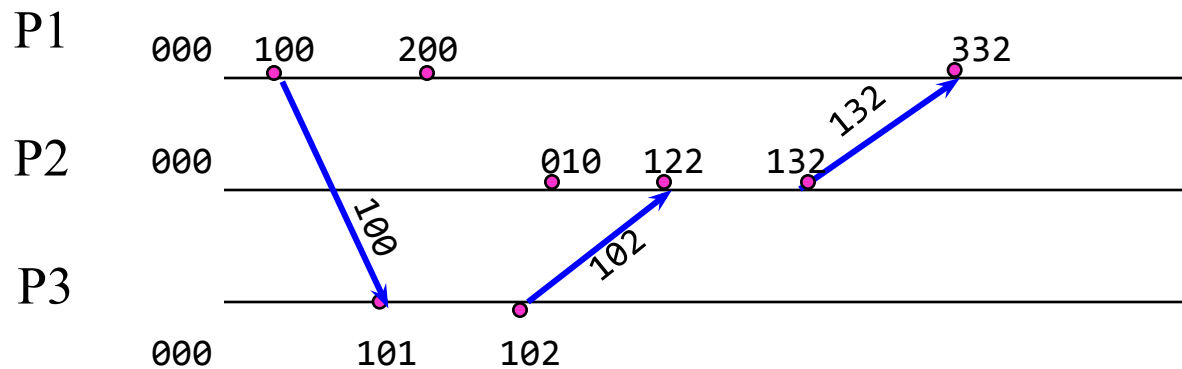
- Evento en P_i
 - evento interno o emisión

$$V_i[i] = V_i[i] + 1$$
 - recepción de un mensaje m con fecha de emisión V_m

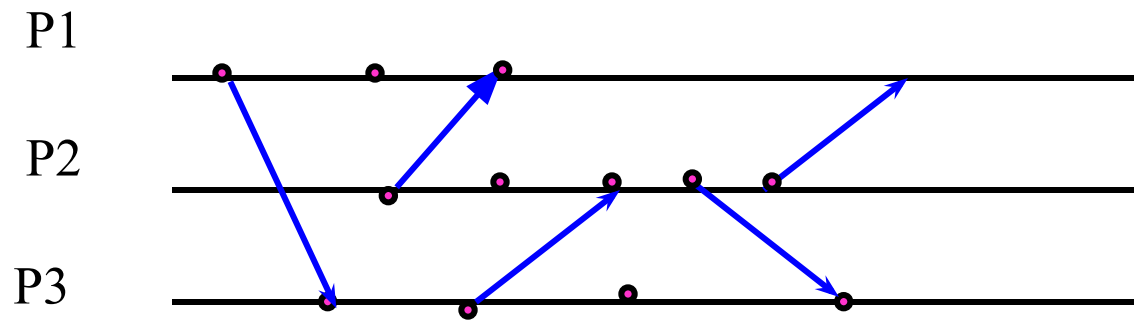
$$V_i[i] = V_i[i] + 1$$

$$V_i[j] = \text{Máx} (V_m[j], V_i[j]) \text{ para toda } j \neq i$$

Orden: Ejemplo relojes vectoriales



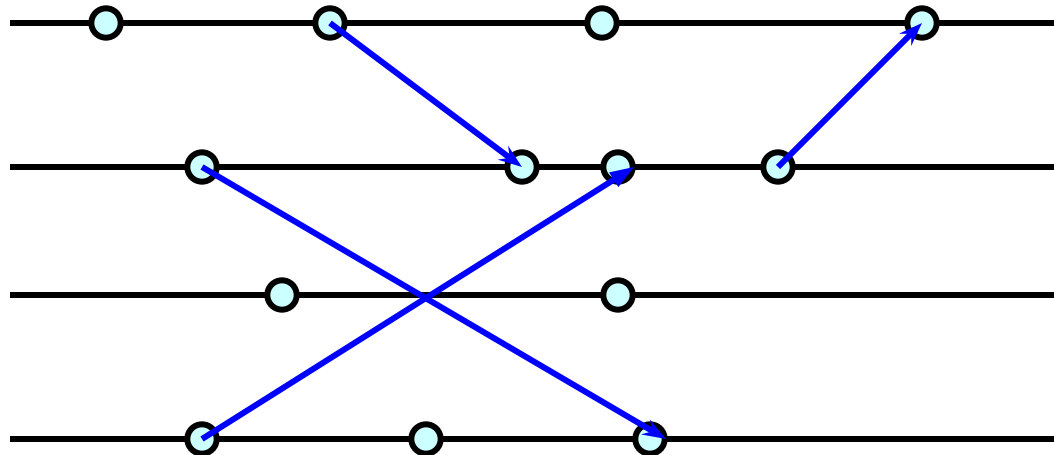
Orden: Ejercicio relojes vectoriales



Dé las fechas lógicas vectoriales de cada uno de los eventos

Relojes vectoriales

- Ejercicio: utilice relojes vectoriales para fechar los eventos en la siguiente ejecución



Relojes vectoriales

- Entre los relojes vectoriales se puede definir la siguiente relación de orden parcial entre ellos:

Sean V y V' dos relojes vectoriales

$$V \leq V' \text{ ssi } \forall i, V[i] \leq V'[i]$$

$$V < V' \text{ ssi } V \neq V' \text{ \& } V \leq V'$$

$$V \parallel V' \text{ ssi } \sim (V < V') \text{ \& } \sim (V' < V)$$

Relojes vectoriales: consistencia fuerte

- ¿Cumplen los relojes vectoriales con la condición de coherencia fuerte? Es decir, dados dos eventos a y b se cumple que:

$$a \rightarrow b \Leftrightarrow V(a) < V(b)$$

Veamos:

a) PD: $a \rightarrow b \Rightarrow V(a) < V(b)$

Si $a \rightarrow b \Rightarrow h(a) \subset h(b)$ y $\forall i \ h_i(a) \subset h_i(b) \Rightarrow$
por construcción $V(a) < V(b)$

b) PD: $V(e) < V'(e') \Rightarrow e \rightarrow e'$ procedemos por reducción al absurdo.

Relojes vectoriales: consistencia fuerte

Supongamos que $V(a) < V(b)$, que $a \parallel b$, que a es un evento que ocurre en P_x y b es un evento que ocurre en P_y .

Sabemos que $h_x(b) = \{e \text{ tal que } e \rightarrow b \text{ y } e \text{ ocurre en } P_x\}$

Sea c el evento mas reciente en $h_x(b)$ entonces:

- ¿ $a \rightarrow c$? **NO** pues entonces $a \rightarrow b$!
- ¿ $a \parallel c$? **NO** pues a y c ocurren ambos en P_x !
- Solo queda la posibilidad que $c \rightarrow a$ lo que implica que

$$V(b)[x] < V(a)[x]$$

- Por un razonamiento simétrico

$$V(a)[y] < V(b)[y]$$

Relojes vectoriales: consistencia fuerte

En consecuencia:

Si $a \parallel b$ entonces $V(a) \parallel V(b)$ dejando como única posibilidad:

$$V(a) < V(b) \Rightarrow a \rightarrow b$$

Y entonces

$$a \rightarrow b \Leftrightarrow V(a) < V(b)$$

Es decir, los relojes vectoriales son fuertemente consistentes.



Ejemplo de uso: Difusión ordenada

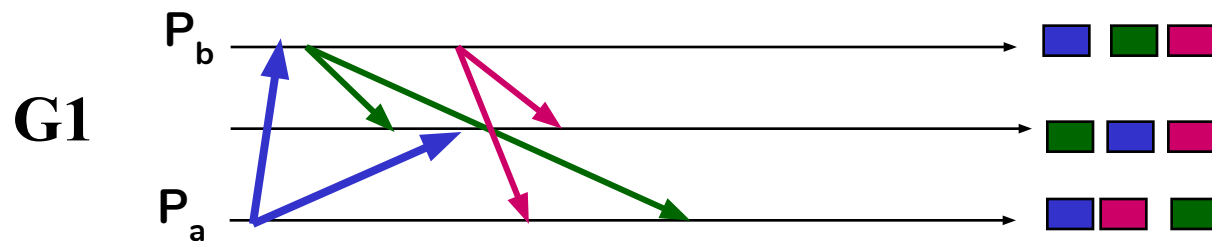
El contexto del problema:

- Se tiene un grupo de procesos comunicantes en donde cada mensaje emitido por uno de los procesos llega a todos los demás.

¿En qué orden reciben los mensajes los miembros del grupo?

Orden

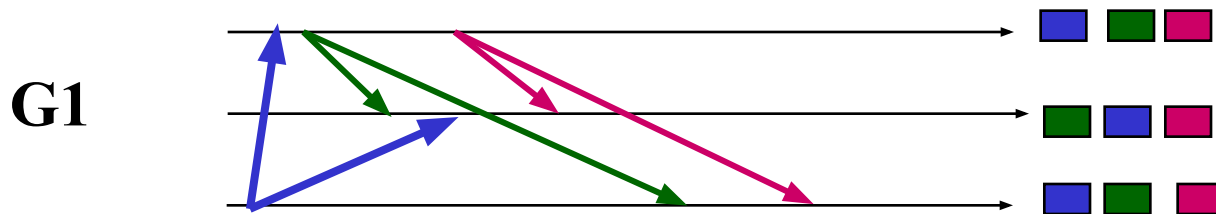
- Sean $G1$ un grupo de procesos y sean m_a el mensaje enviado a $G1$ por P_a y m_b el mensaje enviado a $G1$ por P_b .



¿Cuál orden de recepción es el correcto?

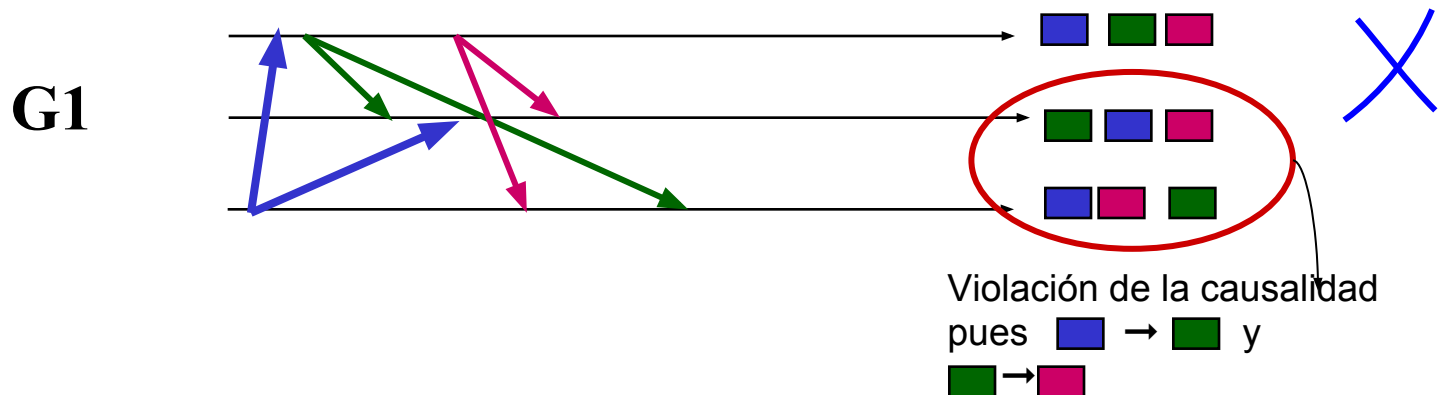
Orden FIFO

- Los mensajes de un emisor son entregados al grupo destinatario en el mismo orden en el que fueron enviados. Esta propiedad es aplicada a los mensajes entre cualquier par emisor-receptor **pero no afecta el orden relativo de mensajes viniendo de distintos emisores.**



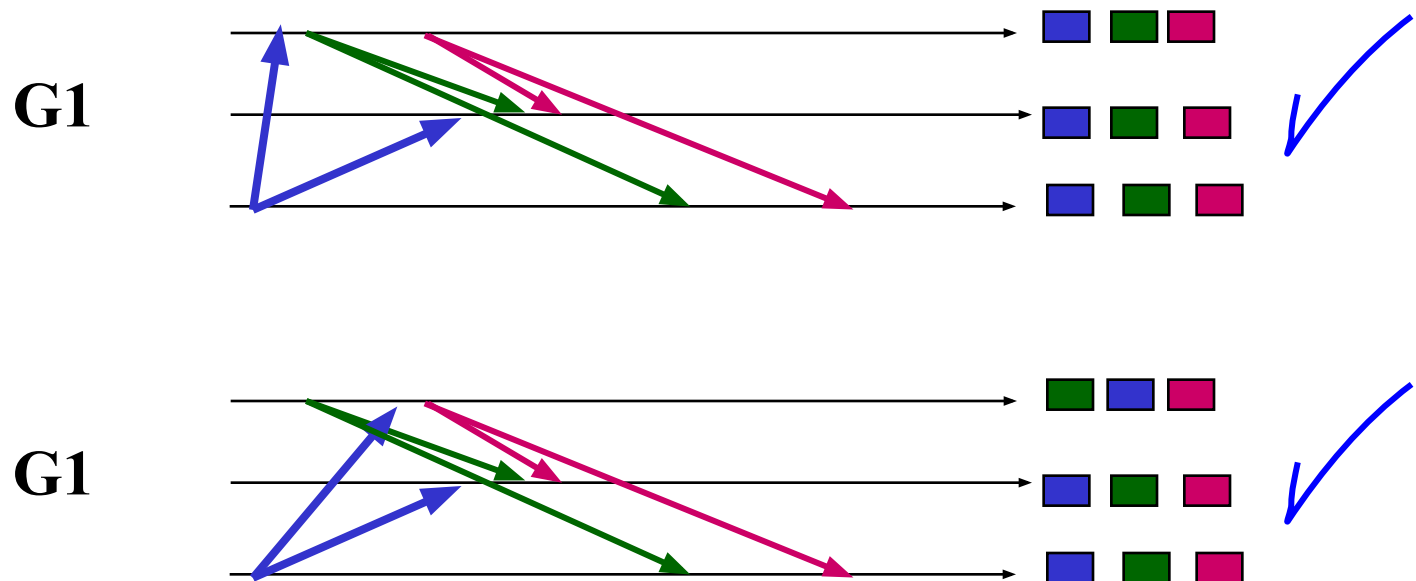
Orden Causal

- Los mensajes son entregados **respetando la relación potencial de causalidad sucedió-antes**. Esta relación representa la posibilidad de que la información llevada en un mensaje tenga influencia sobre la información llevada en otro mensaje.



Orden Causal

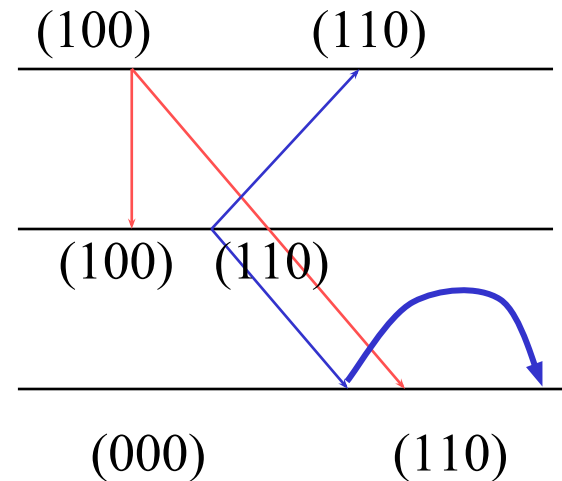
- Si el envío de m_a sucedió antes del envío de m_b entonces m_a es recibido por los demás miembros de G1 antes que m_b .



Algoritmo Causal Broadcast: CBCast

(Birman 90)

- Principio: Se usa un reloj vectorial en cada proceso.
- El reloj vectorial en P_i registra en la entrada i los mensajes enviados por P_i y en la entrada j los mensajes recibidos de P_j



Causal Broadcast: CBCast

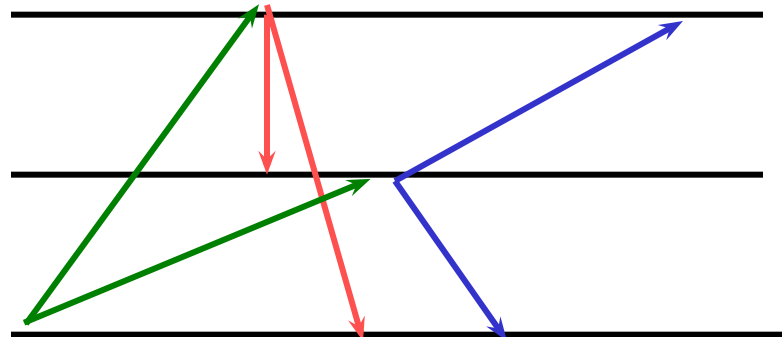
(Birman 90)

- Cada proceso mantiene un reloj vectorial V_i
- Cada que P_i quiere enviar un mensaje primero ejecuta $V_i[i] = V_i[i] + 1$ y asocia al mensaje m la estampilla $V_m = V_i$
- A la recepción de un mensaje m de P_i con estampilla V_m el proceso receptor P_j ($j \neq i$) difiere la entrega hasta que:
 - $V_m[i] = V_j[i] + 1$
 - Para todo $k \neq i : V_m[k] \leq V_j[k]$
- al entregar m , el reloj local se actualiza según
 - $V_j[k] = \max(V_j[k], V_m[k])$ para $k = 1..n$

Causal Broadcast: CBCast

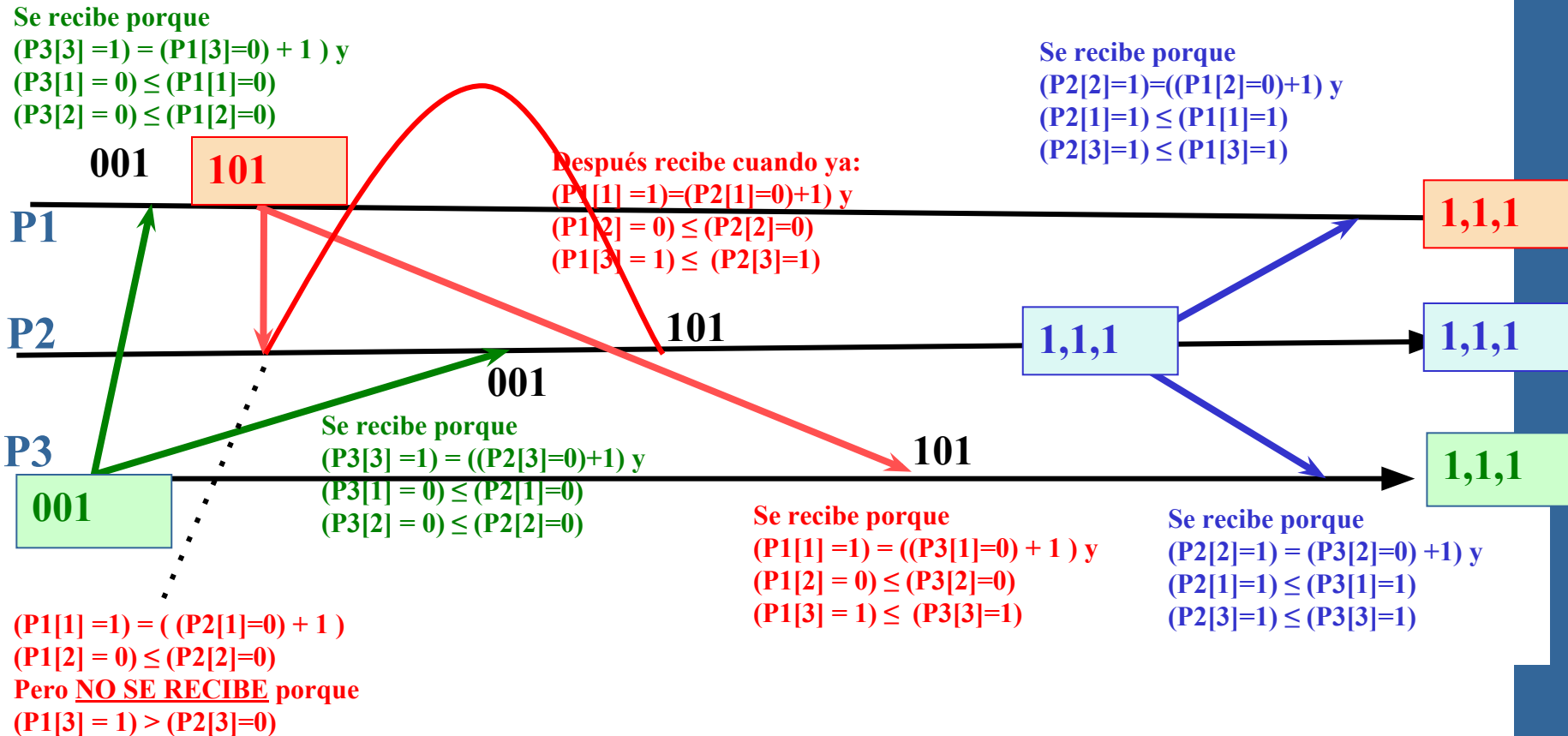
(Birman 90)

Ejemplo de ejecución del algoritmo CBCast sobre la siguiente ejecución.



Causal Broadcast: CBCast

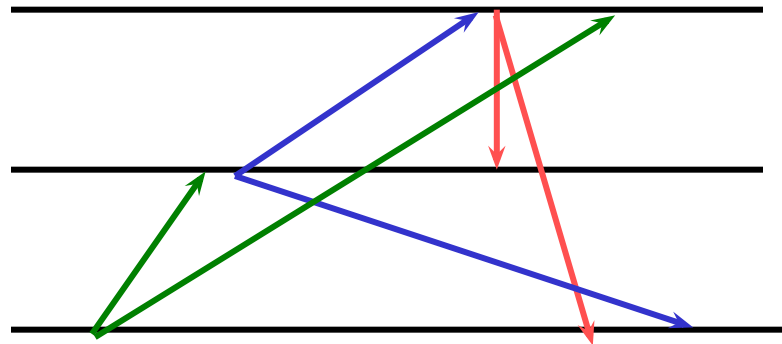
(Birman 90)



Causal Broadcast: CBCast

(Birman 90)

Ejecute el algoritmo CBCast sobre la siguiente ejecución
Muestre las fechas de cada mensaje y el reloj final para cada sitio.



Contenido

4.1 Relojes lógicos escalares

4.2 Relojes lógicos vectoriales

 4.3 Estado global

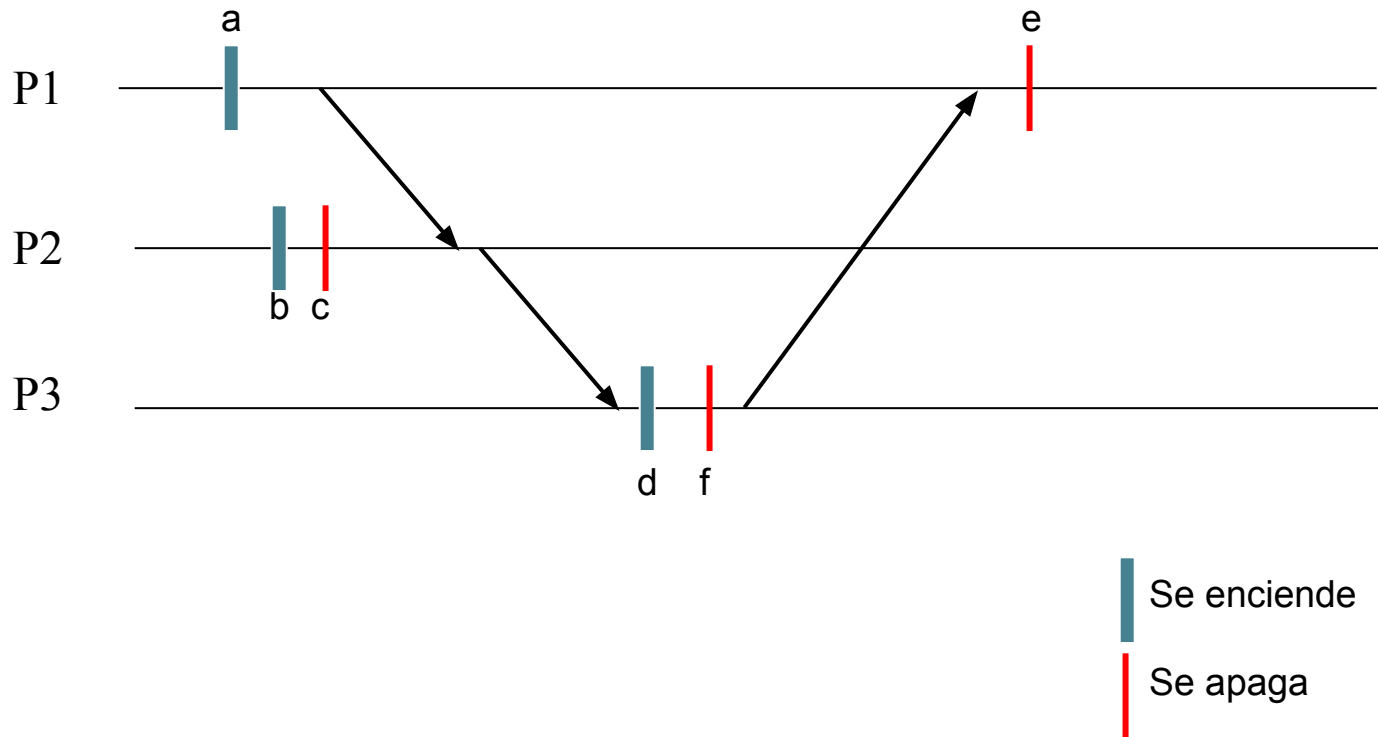
4.3 Estado global

- Recordemos que en los sistemas distribuidos:
 - no hay reloj común
 - ni manera de ver el estado del sistema instantáneamente.

Estado Global

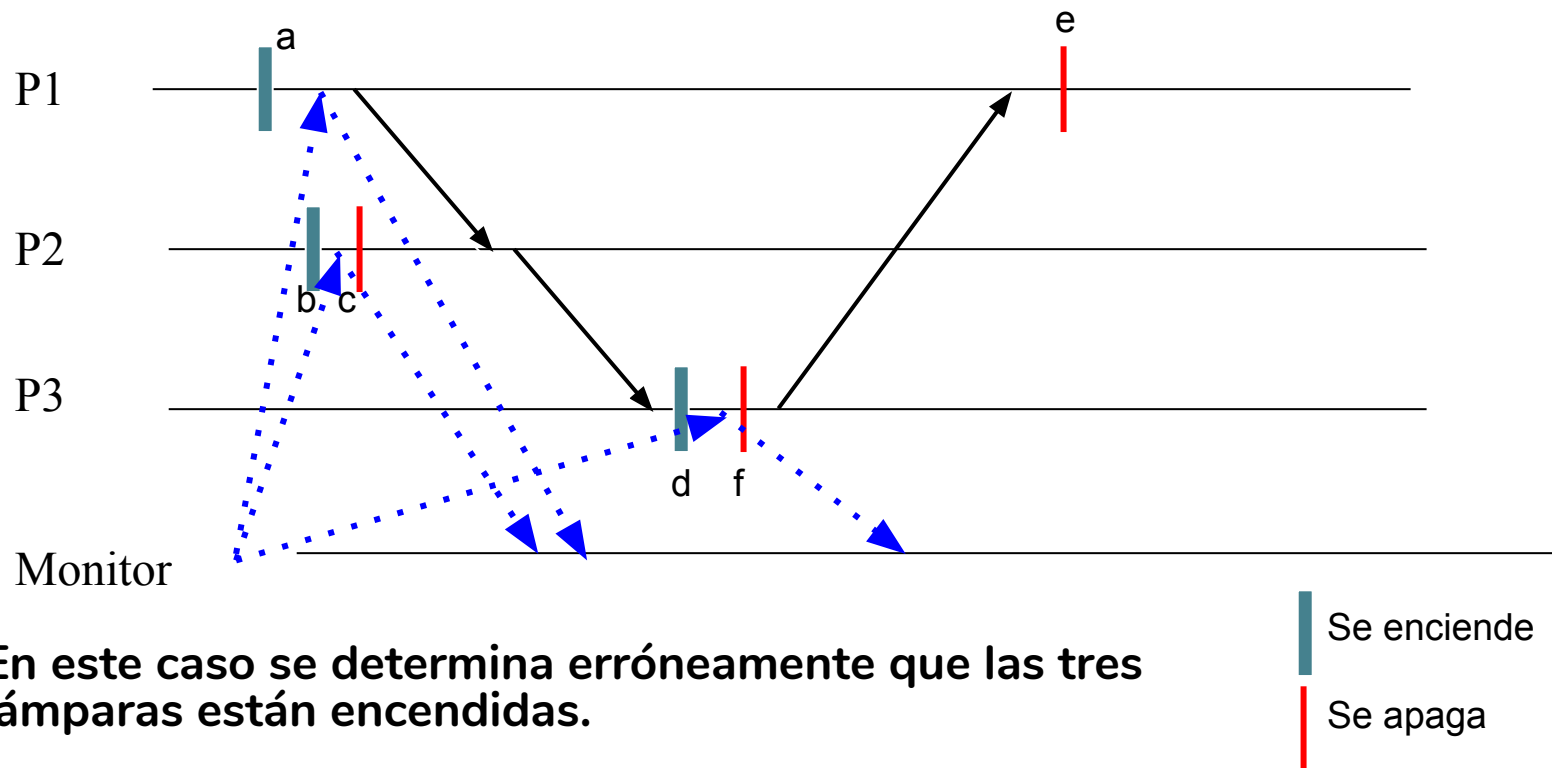
- Necesitamos medios para determinarlo pues es necesario, por ejemplo, para:
 - Registro de estados parciales
 - Depuración distribuida
 - Observación: Detección de propiedades estables (que una vez que son verdaderas permanecen así)
 - Ejemplos: el sistema está interbloqueado, la ejecución del algoritmo terminó, al menos n mensajes han sido enviados...

Ejemplo: ¿en qué estado están estas tres lámparas?



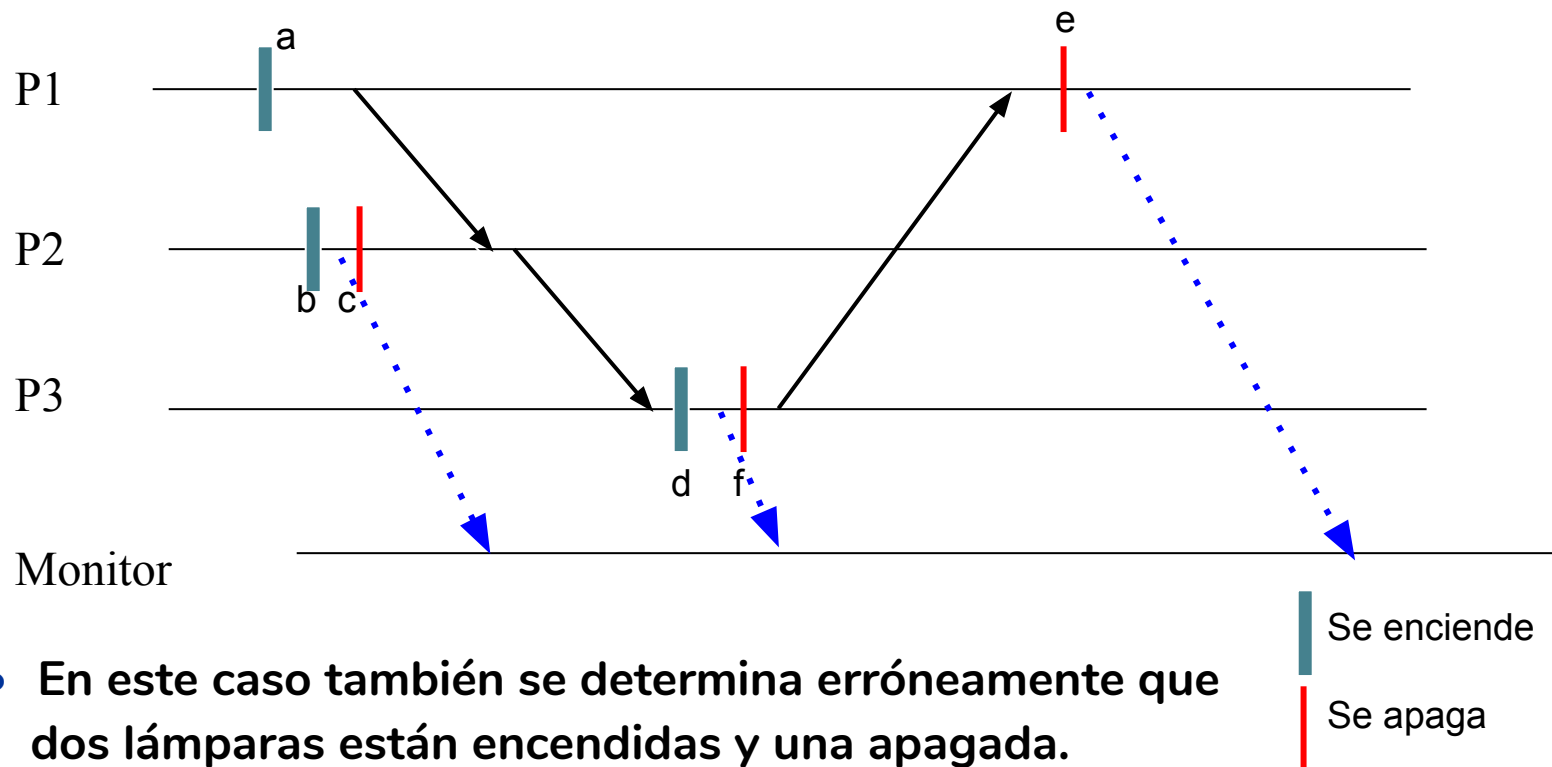
Posibilidades que se nos podrían ocurrir:

- Monitor activo: interroga periódicamente a los dispositivos para determinar su estado



Posibilidades que se nos podrían ocurrir:

- Monitor pasivo: los dispositivos le envían su estado periódicamente



¿Cual es el problema?

- En ambos casos no se preserva la causalidad



Modelo

- Se tiene un conjunto de procesos $\{P1, P2, \dots, Pn\}$
- Cada proceso P_i tiene un estado local σ_i que cambia con cada evento que ocurre en P_i y estos cambios constituyen **la historia local*** de P_i denotada **hi** y representada por una secuencia posiblemente infinita de eventos

$$hi = e^1_i \ e^2_i \ \dots \ e^k_i \ \dots$$

- Sea σ^k_i el estado del proceso P_i después de ejecutar e^k_i y σ^0_i su estado inicial.

* Note que la historia de P_i incluye un orden total entre los eventos



Modelo

- La **historia global H** de una ejecución distribuida está formada por el conjunto de las historias locales h_i de los procesos que la componen.

$$H = h_1 \cup h_2 \cup \dots \cup h_n$$

- Un **estado global** de una ejecución distribuida es una tupla compuesta por un estado local de cada uno de los procesos del sistema:

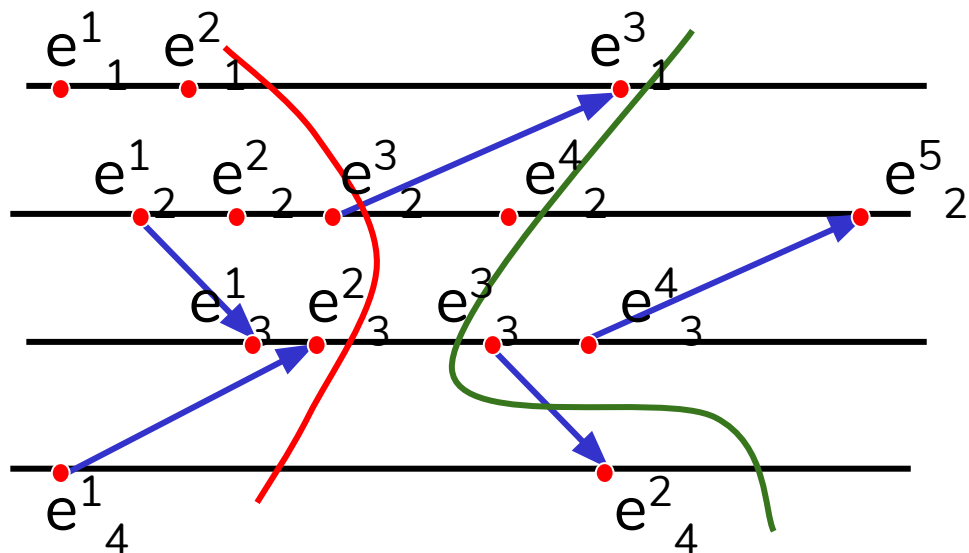
$$\Sigma = \langle \sigma_1, \sigma_2, \dots, \sigma_n \rangle$$



Modelo

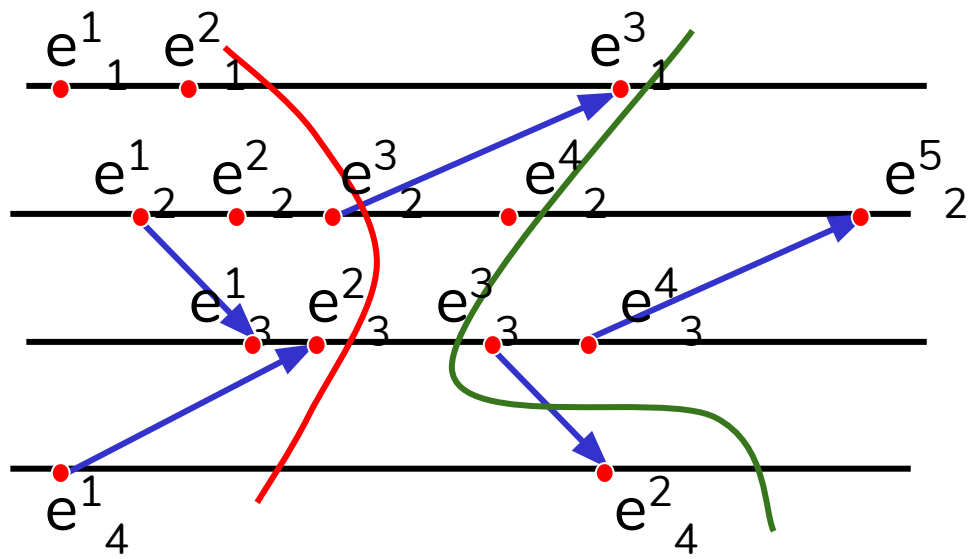
- Un **corte** de una ejecución es un subconjunto de los eventos en H que contiene un prefijo de cada una de las historias locales h_i .

Enseguida presentamos dos cortes:



Modelo

- Se puede especificar un corte mediante la tupla de números naturales $\langle t_1, t_2, \dots, t_n \rangle$ correspondientes a los índices de los últimos eventos ocurridos en cada proceso.



$$C_1 = \langle 2, 3, 2, 1 \rangle$$

$$C_2 = \langle 3, 4, 2, 2 \rangle$$

Modelo

- El corte representa **el estado global del sistema** después de que ocurrieron t_i eventos en el proceso P_i . Esto es, el corte $\langle t_1, t_2, \dots, t_n \rangle$ representa el estado $\langle \sigma^{t_1}_1, \sigma^{t_2}_2, \dots, \sigma^{t_n}_n \rangle$

En nuestro ejemplo:

C1 representa $\langle \sigma^2_1, \sigma^3_2, \sigma^2_3, \sigma^1_4 \rangle$

C2 representa $\langle \sigma^3_1, \sigma^4_2, \sigma^2_3, \sigma^2_4 \rangle$

- El conjunto de los últimos eventos de cada proceso incluidos en el corte $e^{t_1}_1, e^{t_2}_2, \dots, e^{t_n}_n$ se denomina **la frontera del corte**.



Modelo

- Un **corte C es consistente** si para toda pareja de eventos e y e' :

Si $(e \in C)$ y $(e' \rightarrow e)$ entonces $e' \in C$

- Un **estado global consistente** es aquel que corresponde a un corte consistente.
- En nuestro ejemplo:
 - $C1$ y $\langle \sigma^2_1, \sigma^3_2, \sigma^2_3, \sigma^1_4 \rangle$ son consistentes.
 - $C2$ y $\langle \sigma^3_1, \sigma^4_2, \sigma^2_3, \sigma^2_4 \rangle$ no son consistentes pues $(e^2_4 \in C2)$ y $(e^3_3 \rightarrow e^2_4)$ pero $e^3_3 \notin C2$

Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

Modelos: Los usuales con canales FIFO.

Estado global = estado de los sitios + estado de los canales

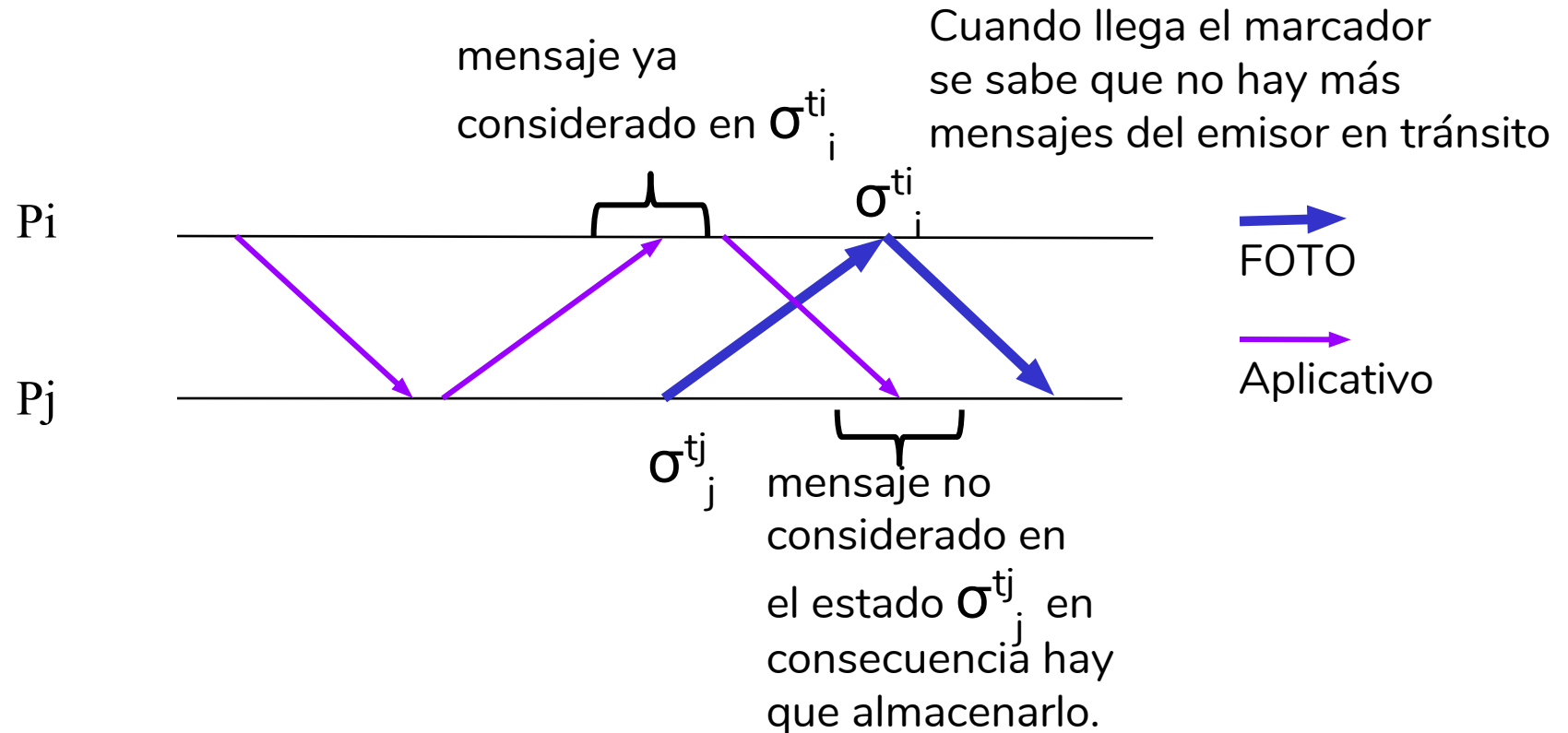
Principio:

- a) Cada proceso que registra su estado envía a los otros un **mensaje marcador (FOTO)** que tiene por objetivo determinar que todos los mensajes que partieron del emisor antes que el marcador ya llegaron.
- b) Un mensaje aplicativo proveniente de un sitio del que no se ha recibido el marcador y recibido después de que el receptor ha guardado su estado local, se almacena en el estado del canal



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)



Algoritmo de Chandy y Lamport

Variables de estado:

- `mi_estado`: almacena el estado del nodo i . Es vacío al inicio.
- `visitado`: indica si el estado local ya se guardó o aún no. Inicialmente es falso.
- `estado_canal[j]`: almacena el estado del canal de j a i . Es $\{\}$ al inicio.
- `pendientes`: indica la cantidad de vecinos que aún no han enviado la indicación de toma de estado global

Algoritmo de Chandy y Lamport

Mensajes:

- FOTO: mensaje marcador enviado por un nodo que ya ha guardado su estado a sus vecinos.
- m: cualquier mensaje aplicativo.



Algoritmo de Chandy y Lamport en P_i

Al recibir INICIA del ambiente haz

visitado = Verdad

mi_estado = valores de todas las variables

salva en disco mi_estado

PARA cada k en vecinos HAZ

Envía FOTO a k

pendientes = |vecinos|



Algoritmo de Chandy y Lamport en P_i

Al recibir FOTO de j HAZ

salva en disco (estado_canal[i-j])

Si visitado = Falso ENTONCES

visitado = Verdad

pendientes = |vecinos|-1

mi_estado = valores de todas las variables

salva en disco mi_estado

PARA cada k en vecinos HAZ

Envía FOTO a k

FINPARA

1



Algoritmo de Chandy y Lamport

Al recibir FOTO de j HAZ ... continua

1

OTRO

pendientes = pendientes - 1

SI (pendientes = 0) ENTONCES

Termina

FINSI



Algoritmo de Chandy y Lamport

Al recibir M desde j HAZ

SI visitado = Verdad ENTONCES

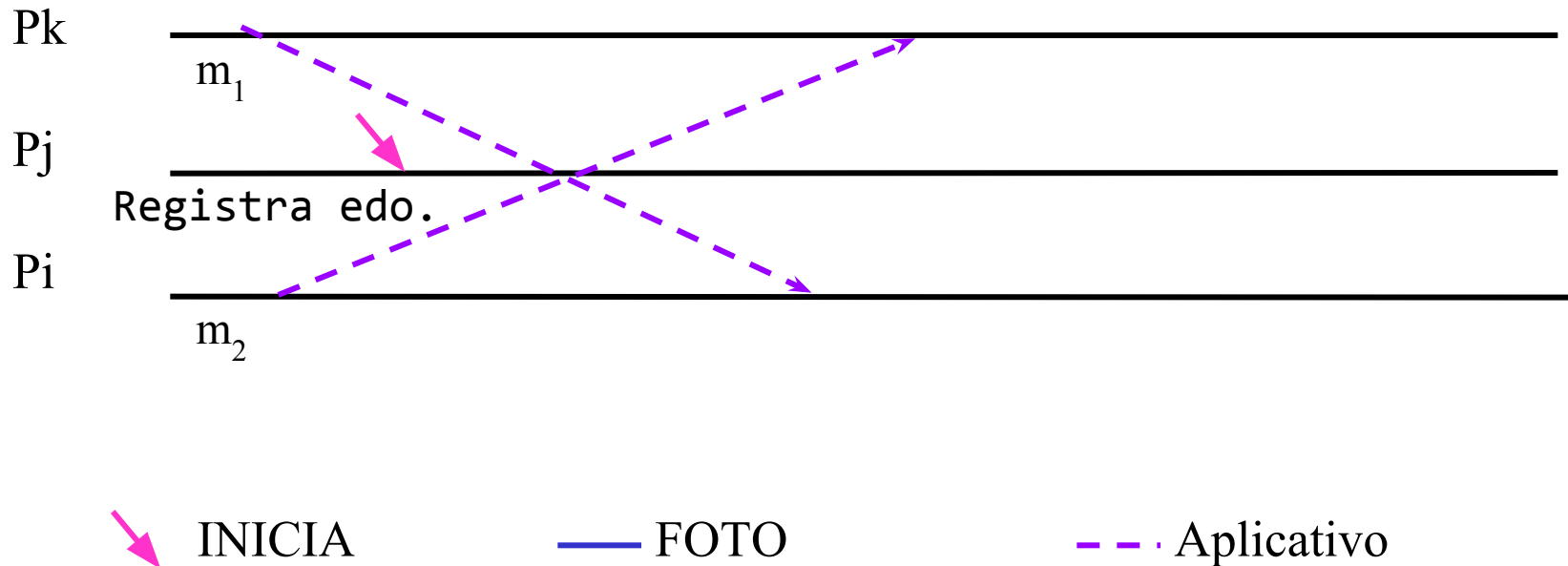
$\text{estado_canal}[j] = \text{estado_canal}[j] \cup \{M\}$



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

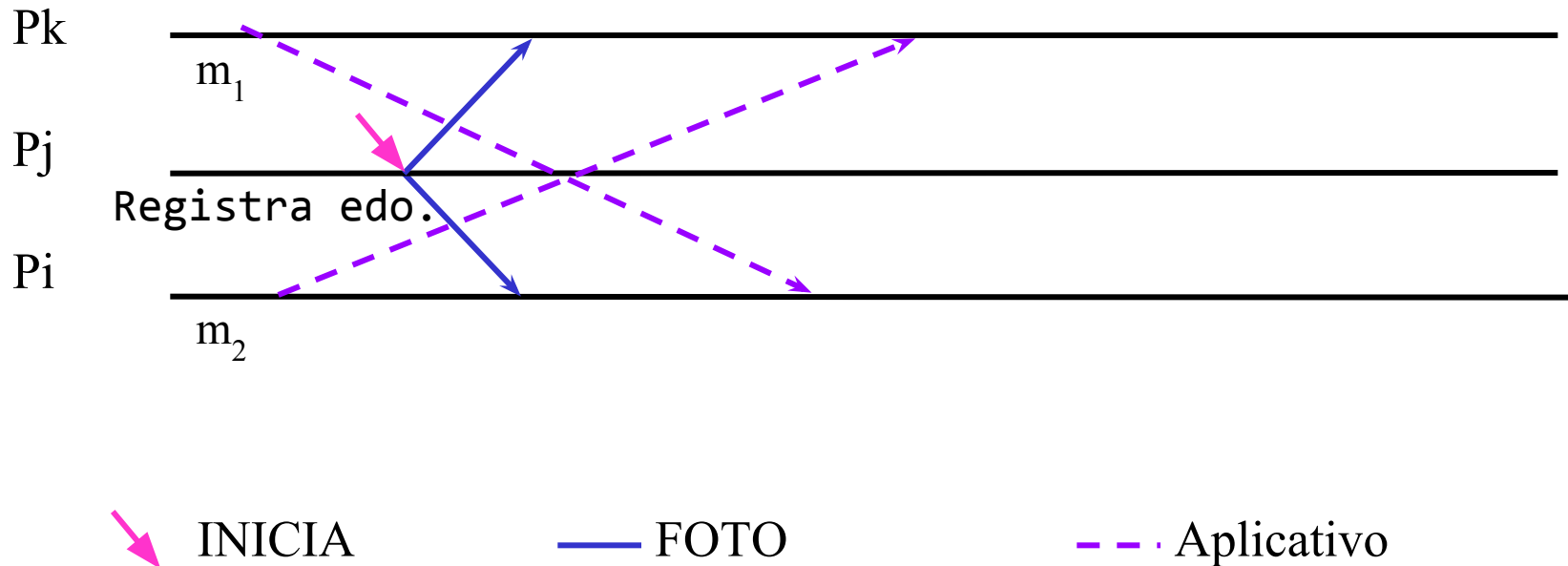
• Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

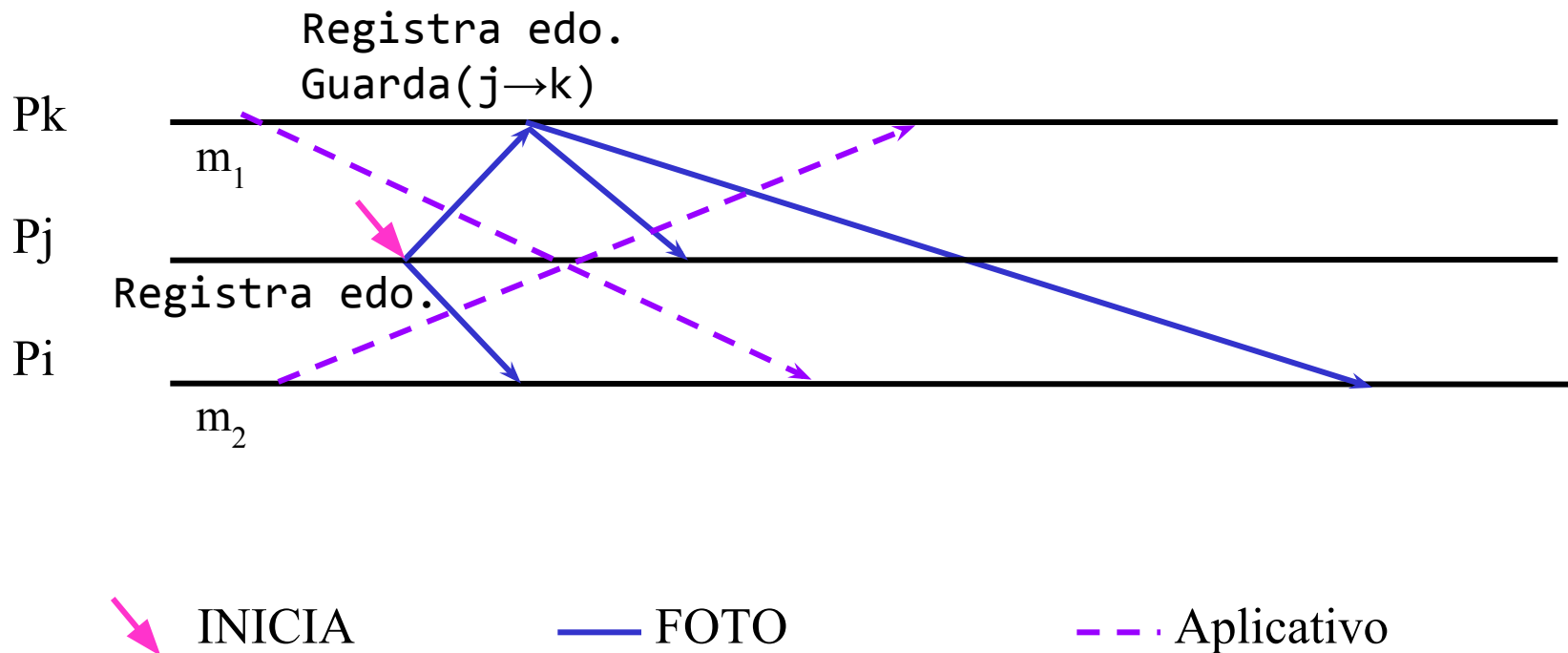
• Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

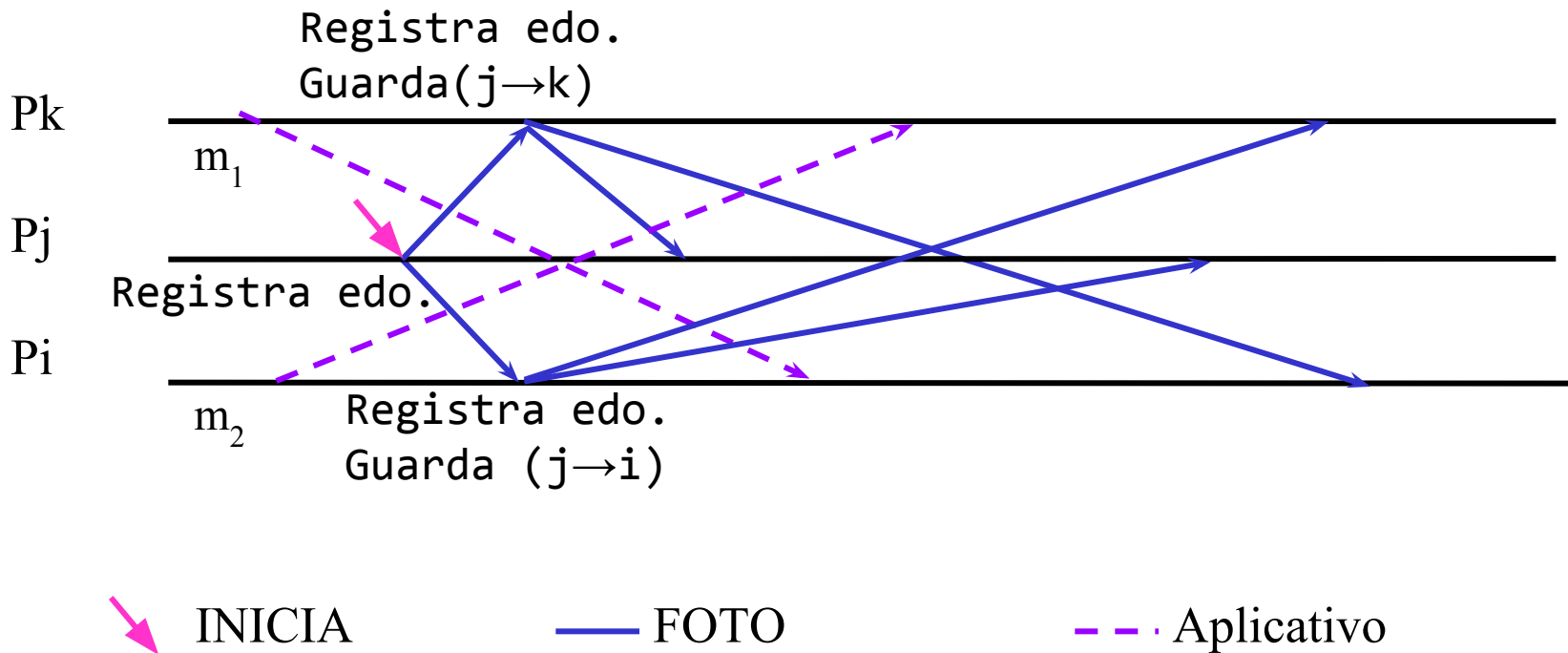
• Ejemplo



(Chandy & Lamport ACM TOCS Feb 1985)

(Chandy & Lamport ACM TOCS Feb 1985)

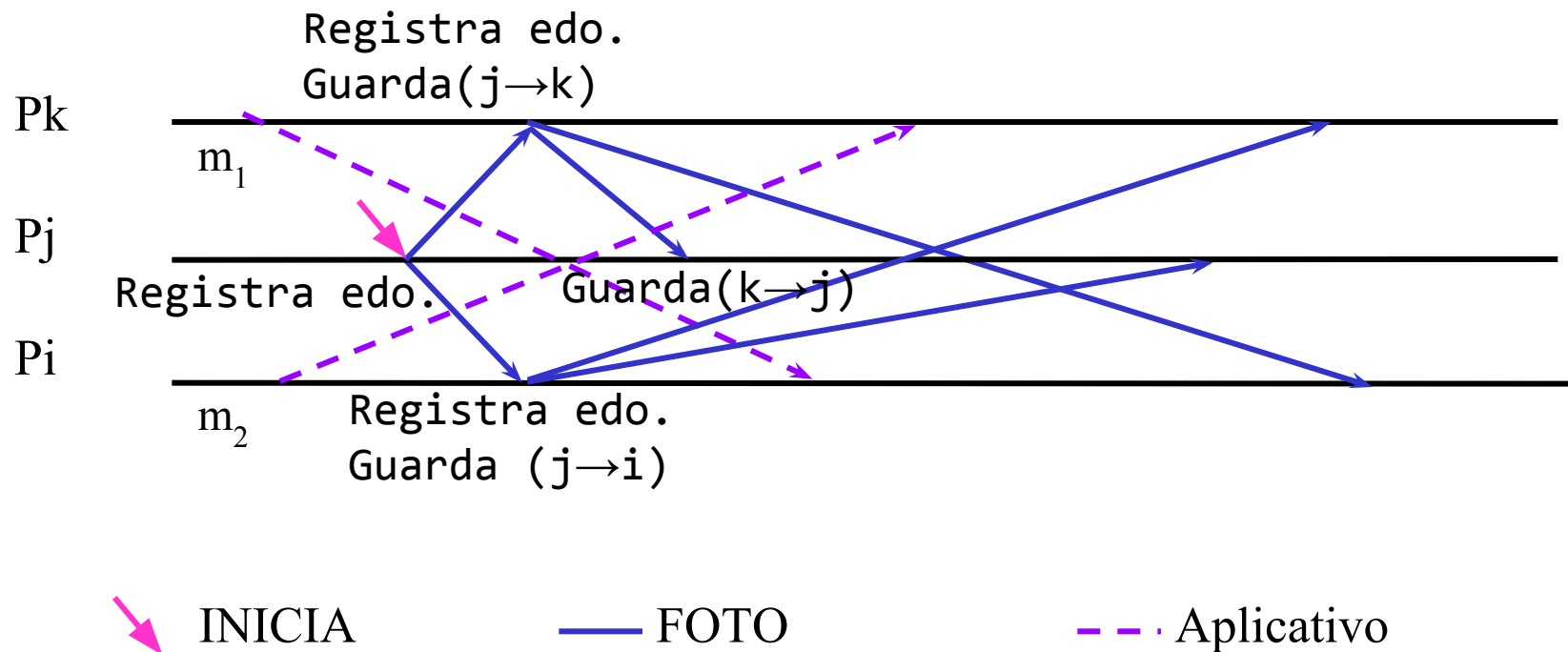
- Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

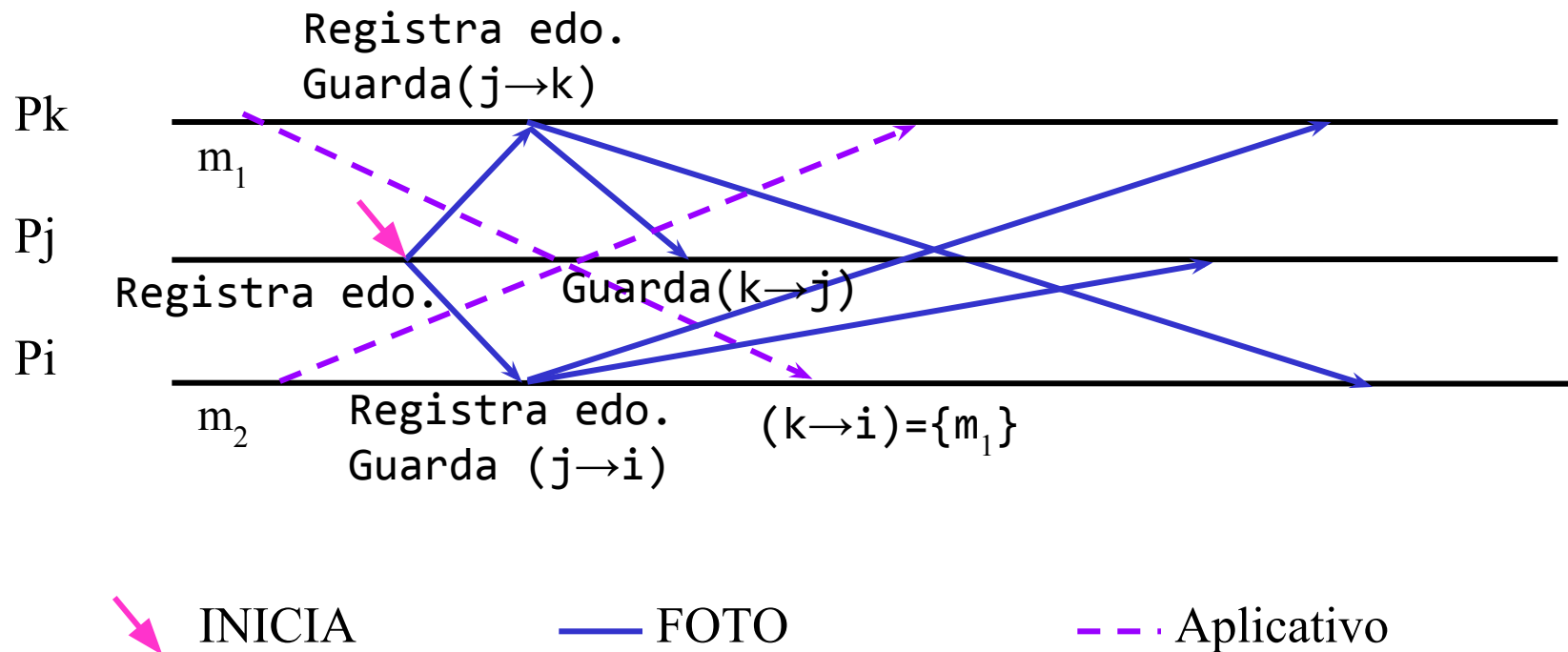
• Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

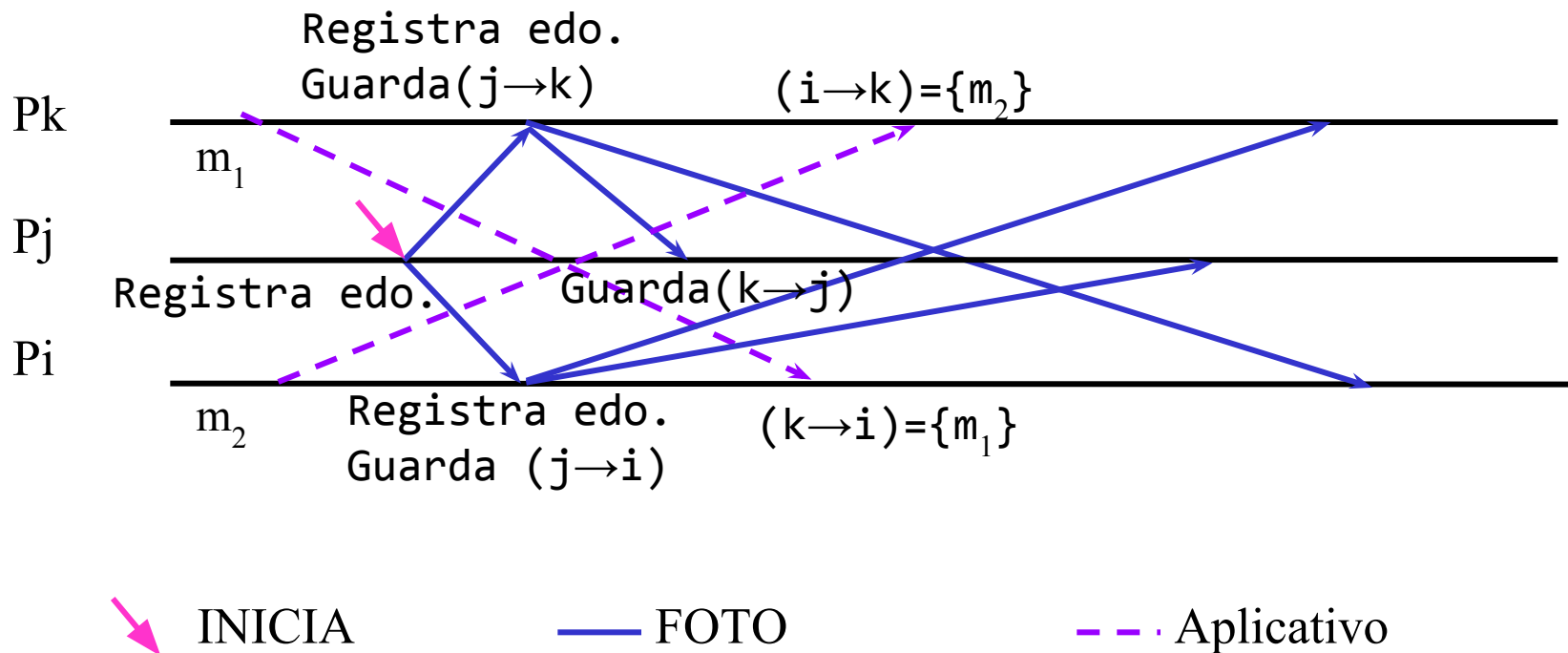
• Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

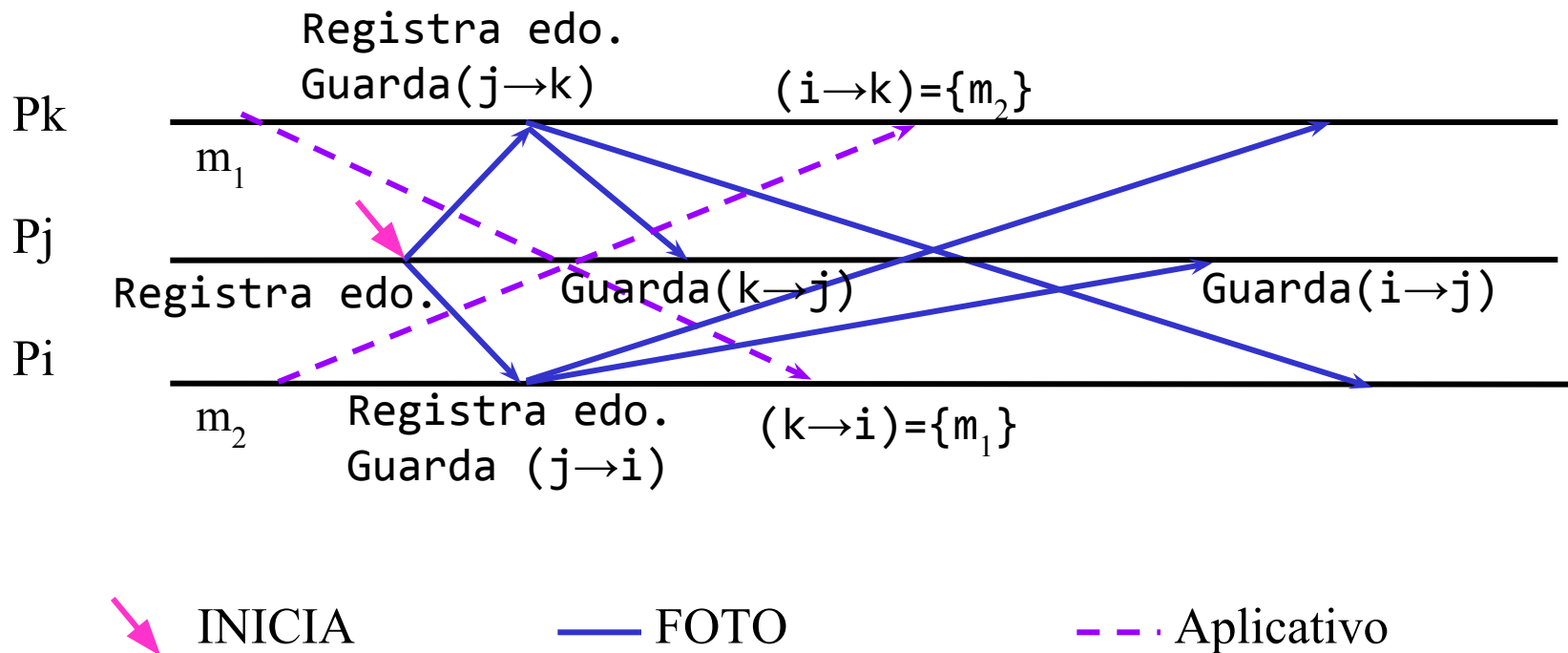
• Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

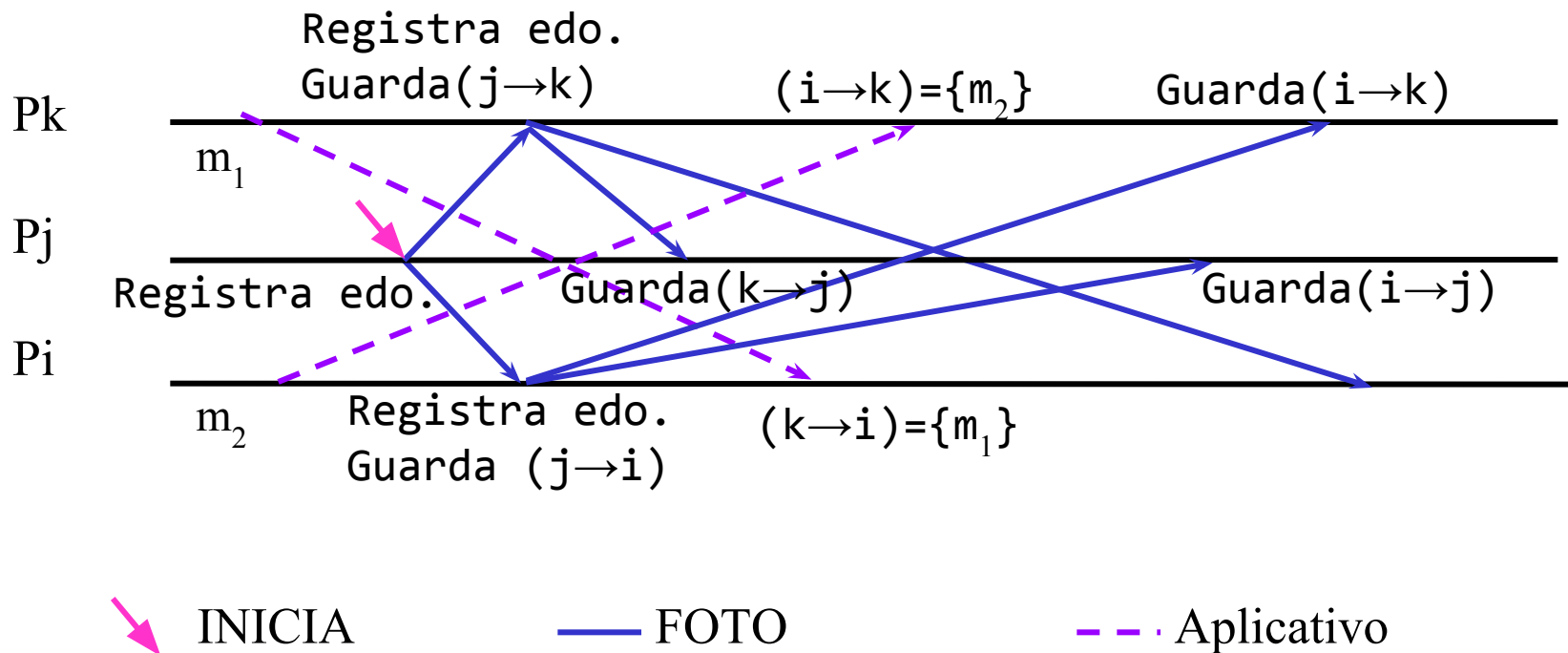
• Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

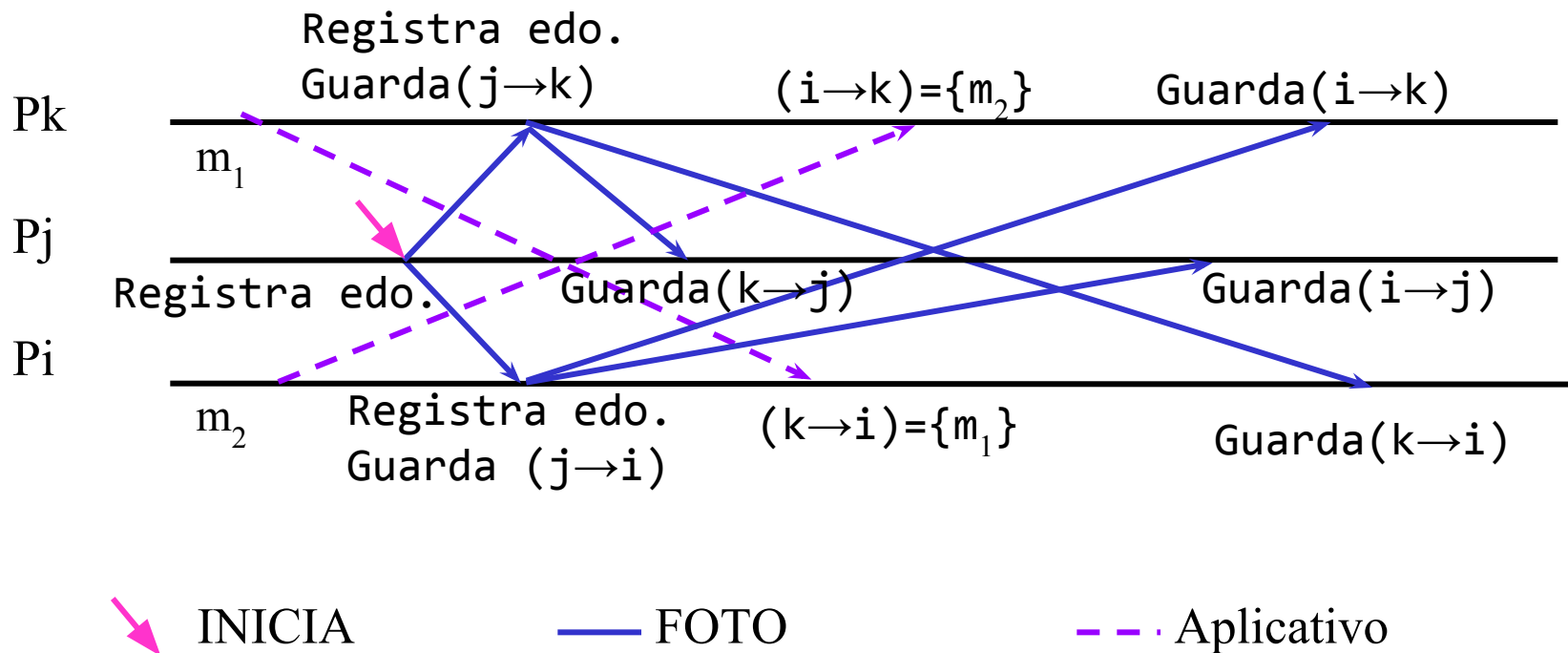
• Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

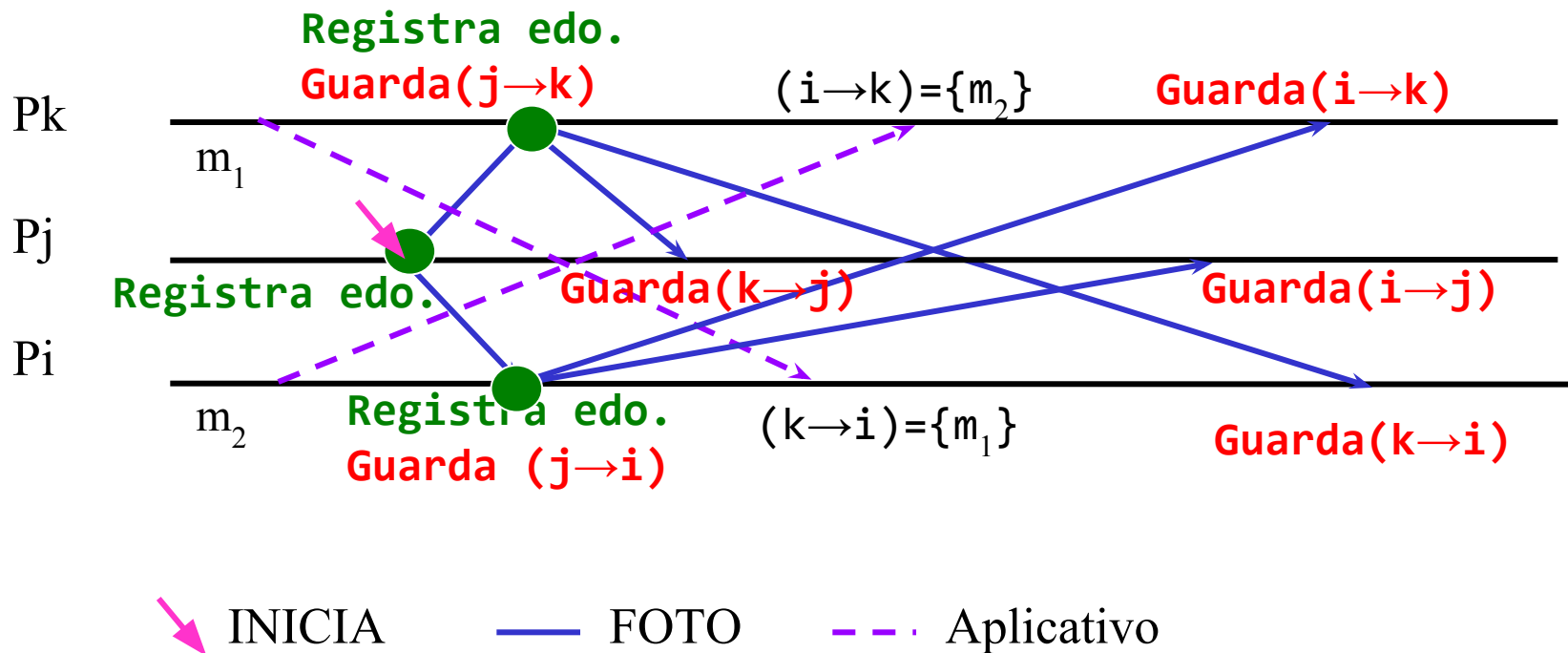
• Ejemplo



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

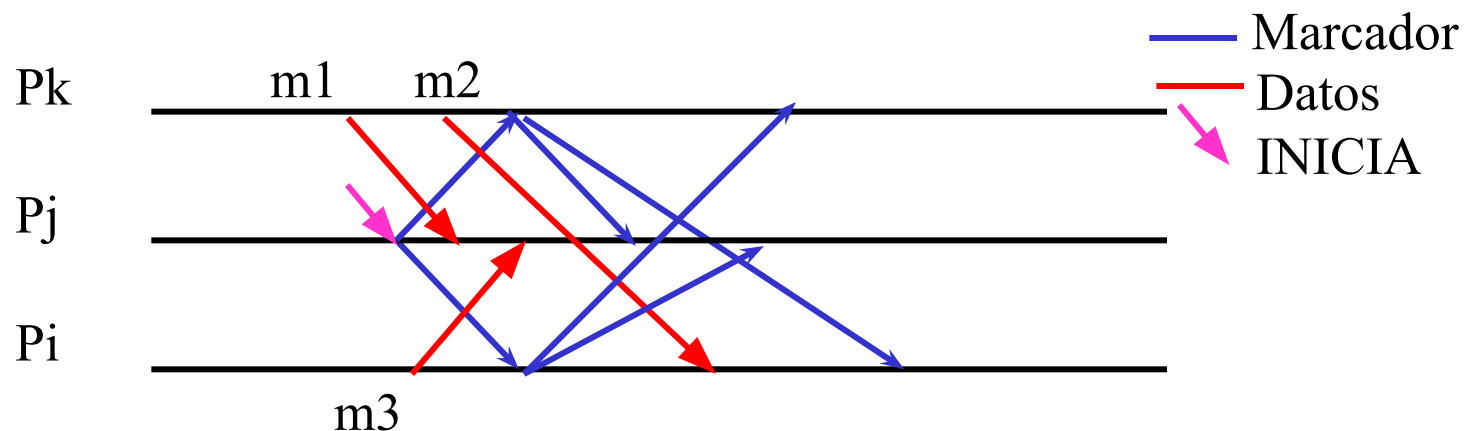
- El estado global está compuesto por los **estados locales almacenados** y el **estado de los canales**.



Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

Ejercicio: diga en qué estado quedan los canales después de la toma de estado global que se presenta

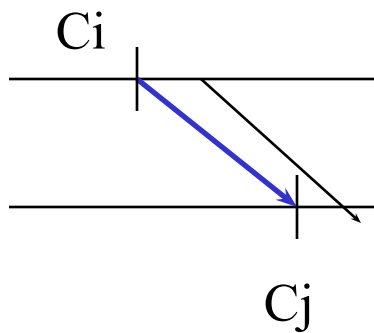


Registro de estado global

(Chandy & Lamport ACM TOCS Feb 1985)

Demostración: El algoritmo colecta un estado global consistente.

Supongamos que existen p_i y p_j tales que el algoritmo tomó un corte incoherente. Es decir, no se registró la emisión de un mensaje pero sí su llegada. Esto no es posible pues



cuando el marcador se envía el estado almacenado contiene todos los mensajes enviados y dado que el canal es FIFO esos mensajes y sólo esos llegan a su destino antes que el marcador.